



UNIVERSIDAD DE TALCA
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA CIVIL EN COMPUTACIÓN

**Sistema IoT de asistencia con autenticación
biométrica e integración empresarial**

AGUSTIN EDUARDO MEZA MELLA

Profesor Guía: RICARDO PÉREZ GUZMÁN

Memoria para optar al título de
Ingeniero Civil en Computación Curicó, Chile— Noviembre, 2026



UNIVERSIDAD DE TALCA
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA CIVIL EN COMPUTACIÓN

**Sistema IoT de asistencia con autenticación
biométrica e integración empresarial**

AGUSTIN EDUARDO MEZA MELLA

Profesor Guía: RICARDO PÉREZ GUZMÁN

El presente documento fue calificado con nota: _____

Memoria para optar al título de
Ingeniero Civil en Computación Curicó, Chile— Noviembre, 2026

TABLA DE CONTENIDOS

Tabla de Contenidos	1
Índice de Figuras	4
Índice de Tablas	5
Resumen	6
1 Introducción	8
1.1 Contexto	8
1.2 Descripción del problema	9
1.3 Propuesta de solución	10
1.4 Objetivos	10
1.4.1 Objetivo general	11
1.4.2 Objetivos específicos	11
1.5 Alcances	12
1.6 Resumen capítulo	13
2 Marco teórico	14
2.1 Conceptos básicos	14
2.1.1 Conceptos globales	14
2.1.2 Conceptos de software	15
2.2 Tecnologías utilizadas	16
2.2.1 Microcontrolador ESP32-CAM	16
2.2.2 Lector de huella digital AS608	16
2.2.3 Protocolos de comunicación HTTP	16
2.2.4 Protocolo de comunicación MQTT	17
2.2.5 Contenedores Docker	17
2.2.6 Sistemas de bases de datos	17
2.3 Estado del arte	18
2.3.1 Soluciones basadas en hardware	18
2.3.2 Soluciones basadas en software	19
2.4 Metodologías	20
2.4.1 Metodología de desarrollo	20
2.4.2 Metodología de evaluación	21
2.5 Resumen del capítulo	22

3	Metodologías	23
3.1	Metodología de desarrollo	23
3.1.1	Protipado Evolutivo Incremental	23
3.2	Planificación	25
3.2.1	Iteración 1: Diseño y configuración inicial	25
3.2.2	Iteración 2: Almacenamiento local y modo offline	26
3.2.3	Iteración 3: Comunicación y conectividad	26
3.2.4	Iteración 4: Detección de rostro y validación biométrica	27
3.2.5	Iteración 5: Módulo antifraude mediante sensor IR	28
3.2.6	Iteración 6: Página web backend e integración con ERP	28
3.2.7	Iteración 7: Validación final del prototipo	29
3.2.8	Iteración 8: Integración total y cierre del proyecto	29
3.3	Metodología de evaluación	30
3.3.1	Pruebas de integración	30
3.3.2	Pruebas de usabilidad	31
3.3.3	Pruebas de confiabilidad y disponibilidad	33
3.4	Resultados Esperados	33
3.5	Resumen del capítulo	34
4	Desarrollo	35
4.1	Diseño de software	35
4.1.1	Arquitectura física	35
4.1.2	Arquitectura lógica	36
4.2	Implementación	36
4.3	Iteración 1: Diseño y planificación inicial	37
4.3.1	Análisis	37
4.3.2	Diseño	38
4.3.3	Implementación	42
4.3.4	Pruebas	50
4.4	Iteración 2: Almacenamiento local y modo offline	51
4.4.1	Análisis	51
4.4.2	Diseño	51
4.4.3	Implementación	52
4.4.4	Pruebas	61
4.5	Iteración 3: Comunicación y conectividad	61
4.5.1	Análisis	62
4.5.2	Diseño	62
4.5.3	Implementación	64

4.5.4	Pruebas	69
4.6	Resumen del capítulo	69

ÍNDICE DE FIGURAS

2.1	Metodología prototipado evolutivo con enfoque incremental	21
4.1	Arquitectura física	35
4.2	Arquitectura lógica	36
4.3	Mockup menú principal en el ESP32-CAM	38
4.4	Mockup panel de configuración Wi-Fi	39
4.5	Mockup registro usuario en el ESP32-CAM	39
4.6	Mockup usuarios almacenados en el ESP32-CAM	40
4.7	Mockup usuarios almacenados en el ESP32-CAM	40
4.8	Mockup creación de turnos en el ESP32-CAM	41
4.9	Mockup asignacion de turnos en el ESP32-CAM	41
4.10	Flujo operacional para el registro de asistencia.	42
4.11	Conexiones entre el ESP32-CAM y el lector de huellas AS608.	43
4.12	Fragmento de código para inicializar el lector AS608 utilizando UART.	44
4.13	asignación de rutas para el servidor web embebido.	45
4.14	Menú principal del sistema web embebido.	46
4.15	Interfaz de configuración Wi-Fi.	46
4.16	Vista para el registro de personas en el dispositivo.	47
4.17	Vista de asistencias locales almacenadas en el dispositivo.	48
4.18	Vista de asistencias locales almacenadas en el dispositivo.	48
4.19	Vista de gestión de turnos locales almacenados en el dispositivo.	49
4.20	Vista de gestión de asignaciones locales almacenadas en el dispositivo.	50
4.21	Inicialización del sistema de archivos SPIFFS.	53
4.22	Búsqueda de slot libre en el lector de huellas AS608.	54
4.23	Procedimiento de enrolamiento biométrico en el AS608.	55
4.24	Registro de usuario y almacenamiento de datos en JSON.	57
4.25	Registro de asistencia en modo offline mediante huella digital.	58
4.26	Registro de asistencia en modo offline mediante huella digital.	59
4.27	Funciones para lectura y escritura de archivos JSON en SPIFFS.	60
4.28	Diagrama preliminar de bases de datos	64
4.29	Función de conexión WiFi en el ESP32-CAM	65
4.30	Inicialización de la camara del ESP32-CAM	65
4.31	Envío de imágenes por partes desde el ESP32-CAM	66
4.32	Método para la reconexión con el MQTT	67
4.33	Funcion para recibir imagenes en el backend.	67
4.34	Metodo generico para envio de peticiones HTTP.	69

ÍNDICE DE TABLAS

RESUMEN

El presente proyecto aborda el desarrollo de un sistema IoT de control de asistencia biométrica, diseñado para responder a las limitaciones actuales de las soluciones comerciales y cumplir con los requisitos técnicos y legales establecidos por la Dirección del Trabajo de Chile, en particular la Resolución Exenta N.º38 (2024), que regula los sistemas electrónicos de registro de asistencia. El trabajo propone una alternativa modular, de bajo costo, código abierto y adaptable, orientada principalmente a pequeñas y medianas empresas que requieren un sistema flexible, con capacidad de funcionamiento autónomo y con posibilidades reales de integración con plataformas ERP.

La solución desarrollada utiliza como dispositivo central un ESP32-CAM, microcontrolador que incorpora cámara, conectividad Wi-Fi y capacidades de procesamiento suficientes para realizar captura de imágenes y ejecutar tareas de apoyo asociadas a la autenticación. A este módulo se integra un lector de huellas AS608, que permite una verificación biométrica complementaria frente a situaciones donde la captura facial no sea óptima. El sistema se estructura de manera híbrida, permitiendo registrar asistencia tanto mediante reconocimiento facial como mediante verificación dactilar, de acuerdo con las condiciones de operación.

Uno de los principales aportes del proyecto es la capacidad del dispositivo para operar en modo offline, utilizando SPIFFS como sistema de almacenamiento local para gestionar usuarios, turnos, asignaciones y asistencias en formato JSON. Cuando la conectividad se restablece, el sistema ejecuta un proceso de sincronización diferida que garantiza la integridad de los datos y la continuidad operativa del dispositivo, incluso en entornos con conectividad inestable o intermitente, situación común en empresas pequeñas o en faenas remotas. Asimismo, la arquitectura lógica y física diseñada permite incorporar módulos de seguridad adicionales, como un sensor IR antifraude, destinado a detectar intentos de suplantación mediante fotografías o pantallas.

El desarrollo del proyecto se llevó a cabo utilizando una metodología de prototipado evolutivo incremental, seleccionada por su capacidad para integrar hardware, firmware, backend y módulos biométricos de manera progresiva. A lo largo de ocho iteraciones se construyeron y validaron componentes funcionales independientes que luego fueron integrados: capturas biométricas, almacenamiento local, comunicación HTTP/MQTT, panel web embebido, mecanismos antifraude, backend con base de datos PostgreSQL y módulo de comunicación estándar para ERP. Cada iteración incluyó análisis, diseño, implementación y pruebas, permitiendo detectar fallas tempranas y optimizar el prototipo de manera continua.

La etapa de evaluación incorporó pruebas de integración, usabilidad, confiabilidad y disponibilidad, fundamentales para determinar el desempeño del sistema en escenarios

reales. Las pruebas de integración verificaron la correcta comunicación entre los distintos módulos del prototipo, incluyendo la interoperabilidad entre el ESP32-CAM, el lector AS608, el almacenamiento SPIFFS, el backend y la base de datos. Las pruebas de usabilidad, aplicadas tanto a usuarios técnicos como no técnicos mediante el método System Usability Scale (SUS), permitieron evaluar la claridad de las interfaces y la facilidad de uso del sistema. Complementariamente, se desarrollaron pruebas de confiabilidad y disponibilidad destinadas a medir la robustez del sistema frente a desconexiones, reconexiones y funcionamiento prolongado sin supervisión.

El sistema resultante constituye un prototipo funcional que permite registrar asistencia, almacenar datos localmente, sincronizar registros con un backend cuando existe conectividad, operar de manera continua bajo distintos escenarios y comunicar información hacia sistemas ERP mediante una API estándar. A través del análisis de los resultados obtenidos, se concluye que la solución propuesta es técnicamente viable, cumple con los requisitos de funcionamiento planteados en los objetivos específicos y representa una alternativa accesible y adaptable frente a las restricciones de los dispositivos biométricos comerciales actuales.

1. Introducción

En el siguiente capítulo se presenta el contexto, la problemática actual y la propuesta de solución que fundamentan el desarrollo de este sistema IoT de control de asistencia biométrico.

1.1 Contexto

El control de asistencia laboral es un requisito obligatorio para todas las empresas en Chile, conforme a lo dispuesto por la Dirección del Trabajo. Este registro puede realizarse de manera manual o mediante sistemas digitales que cumplan con la Resolución Exenta N°38 (2024) [1], la cual establece los requisitos técnicos y legales que deben cumplir los sistemas electrónicos para ser considerados válidos ante la autoridad.

El cumplimiento de estas normas no solo busca garantizar la trazabilidad y transparencia de las jornadas laborales, sino que también impacta directamente en el cálculo de remuneraciones y en la gestión administrativa de la empresa[2], si no se lleva un registro que cumpla las características presentes en la norma, tales como la inclusión de métodos que garanticen el registro de asistencia al trabajador. Por ello, muchas organizaciones optan por integrar sus sistemas de asistencia con plataformas ERP (EntERPrise Resource Planning), las cuales centralizan distintos procesos como recursos humanos, inventario, finanzas y contabilidad dentro de un mismo entorno[3].

Esta integración permite que los datos de asistencia alimenten automáticamente los módulos de remuneraciones o de gestión de personal, evitando errores y reduciendo los tiempos administrativos[4]. En este contexto, los sistemas de asistencia pueden ser parte del propio ERP o soluciones externas especializadas, las cuales se comunican mediante APIs o servicios web.

Las máquinas de control de asistencia digital, por su parte, permiten registrar las marcaciones de entrada y salida de los trabajadores de forma autónoma[5]. Sin embargo, muchas de las soluciones disponibles en el mercado presentan limitaciones: operan únicamente en red local, exportan datos en formatos cerrados (como CSV o Excel) y

no siempre permiten integrarse de manera flexible con otros sistemas, como los ERP corporativos.

Este proyecto propone el desarrollo de un sistema IoT de control de asistencia basado en ESP32-CAM, capaz de realizar reconocimiento facial y enviar los registros al backend, permitiendo su posterior integración con un ERP o funcionamiento independiente en modo local.

1.2 Descripción del problema

Aunque la normativa chilena exige que los sistemas de control de asistencia garanticen trazabilidad, integridad y disponibilidad de los registros de acuerdo con la Resolución Exenta N.º 38 (2024), una parte importante de los dispositivos utilizados actualmente especialmente los relojes de control biométricos de bajo costo presentan limitaciones que dificultan su adopción en empresas pequeñas y medianas. Muchos de estos equipos requieren conexión permanente a internet, operan con software propietario o solo permiten exportar datos en formatos estáticos como CSV, lo que genera dependencia de procesos manuales y restringe la interoperabilidad con otros sistemas.

Al mismo tiempo, la mayoría de los ERP utilizados por las empresas no incorpora módulos nativos para capturar asistencia desde hardware, por lo que su integración con dispositivos externos resulta costosa o directamente incompatible. La ausencia de APIs abiertas, la falta de mecanismos de sincronización automática y la poca flexibilidad para adaptar el hardware al contexto operativo dificultan la incorporación de tecnología biométrica en entornos reales.

Estas limitaciones se vuelven más evidentes en escenarios donde la conectividad es inestable. Muchos relojes de control no pueden almacenar marcaciones localmente de manera segura ni realizar una sincronización diferida una vez restablecida la red, lo que expone a las empresas a la pérdida de datos y al incumplimiento normativo. asimismo, la mayoría de las soluciones comerciales no permite personalizar la lógica de validación biométrica, integrar módulos anti fraude ni extender el sistema a necesidades particulares, como gestión de turnos o funcionamiento híbrido.

Actualmente no existe en el mercado una alternativa accesible, modular y adaptable que permita capturar asistencia mediante biometría (huella o reconocimiento facial), operar de forma autónoma sin internet, almacenar registros en el dispositivo, detectar intentos de suplantación y sincronizar datos con sistemas ERP mediante APIs estándar. Esta brecha tecnológica afecta especialmente a organizaciones que requieren una solución flexible, de bajo costo y consistente con las exigencias de la autoridad laboral.

En síntesis, el problema consiste en la falta de un reloj control biométrico capaz de operar offline, almacenar y sincronizar marcaciones de manera segura, y conectarse a

sistemas ERP mediante interfaces estándar cumpliendo la normativa vigente.

1.3 Propuesta de solución

La propuesta consiste en diseñar e implementar un prototipo de control de asistencia utilizando un módulo ESP32-CAM junto con un lector de huellas dactilares. El ESP32-CAM es una placa de bajo costo que incorpora una cámara, conectividad Wi-Fi y Bluetooth, lo que permite capturar imágenes, procesarlas y comunicarse con otros sistemas sin necesidad de equipos adicionales.

El objetivo es que el dispositivo pueda registrar la asistencia de los trabajadores mediante reconocimiento facial y huella digital, funcionando tanto con conexión a internet como sin ella. En modo offline, el equipo almacenará temporalmente los registros en una tarjeta microSD o en su memoria interna, lo que le permitirá operar de manera autónoma durante varios días o incluso semanas, dependiendo de la capacidad de almacenamiento configurada. Cuando se recupere la conexión, los datos se sincronizarán automáticamente con un servidor web o con un sistema ERP, según la configuración establecida por el usuario.

El lector de huellas permitirá complementar la verificación facial, ofreciendo una alternativa en lugares donde las condiciones de luz o el ángulo de la cámara dificulten el reconocimiento.

El prototipo contará además con una interfaz web integrada, accesible desde la misma red local, que permitirá configurar parámetros de red, administrar usuarios y definir el destino de los datos. Junto con ello, se desarrollará una documentación técnica de apoyo, donde se describirá el funcionamiento general del sistema, el formato de los registros generados y las opciones de integración con diferentes plataformas ERP. Esta integración se realizará mediante un endpoint o API estándar, lo que permitirá la conexión con cualquier sistema ERP que acepte intercambio de datos a través de solicitudes HTTP o JSON, brindando flexibilidad sin depender de un proveedor específico.

Finalmente, para mejorar la seguridad en el proceso de marcación, se incorporarán métodos que eviten la suplantación de identidad, impidiendo que se utilicen fotografías o imágenes impresas para registrar asistencias falsas.

1.4 Objetivos

En el siguiente apartado se procederá a presentar el objetivo general del proyecto, junto con los objetivos específicos.

1.4.1 Objetivo general

Probar la utilidad de un sistema de control de asistencia IoT de bajo costo y código abierto, desarrollado con un ESP32-CAM, que permita optimizar la gestión de asistencia laboral en pequeñas y medianas empresas mediante integración con ERP y funcionamiento offline.

1.4.2 Objetivos específicos

Objetivo específico 1

Diseñar la arquitectura general del sistema IoT, definiendo componentes y flujos de datos.

Objetivo específico 2

Implementar el módulo biométrico de registro de asistencia mediante reconocimiento facial y huella digital.

Objetivo específico 3

Incorporar capacidad de funcionamiento offline con almacenamiento local y sincronización posterior.

Objetivo específico 4

Desarrollar una interfaz web embebida para la configuración y administración del dispositivo.

Objetivo específico 5

Implementar una API de comunicación para la integración con servidores o sistemas ERP externos.

Objetivo específico 6

Integrar y validar el funcionamiento conjunto del hardware y software desarrollado.

Objetivo específico 7

Evaluar el desempeño, confiabilidad y usabilidad del sistema en escenarios de prueba reales.

Objetivo específico 8

Analizar los resultados obtenidos para determinar la viabilidad.

1.5 Alcances

A continuación se presentarán los alcances propuestos para este proyecto.

- Se diseñará la arquitectura de hardware y software del sistema, incluyendo los módulos de captura de imagen, registro de huella, almacenamiento local y comunicación con el servidor.
- Se implementará un prototipo funcional capaz de registrar asistencias mediante reconocimiento facial y huella digital, operando en modo offline con almacenamiento local y sincronización posterior.
- Se desarrollará una interfaz web embebida para la configuración del dispositivo, administración de usuarios y definición del destino de los datos.
- Se integrará una API RESTful para permitir la comunicación entre el dispositivo y un servidor o sistema ERP, utilizando formatos de intercambio JSON sobre HTTP.
- Se incorporarán medidas básicas de seguridad biométrica orientadas a evitar la suplantación de identidad mediante imágenes o fotografías.
- Se realizarán pruebas de funcionamiento offline, midiendo la autonomía del sistema (tiempo máximo de operación sin conexión y capacidad de almacenamiento).
- Se ejecutarán pruebas de integración entre los módulos de hardware, software y el ERP simulado, verificando el flujo completo de datos desde el registro hasta la sincronización.
- Se aplicarán pruebas de usabilidad para evaluar la facilidad de configuración y el nivel de satisfacción del usuario final con la interfaz embebida.
- Se efectuarán pruebas de confiabilidad e integridad, comprobando que los datos registrados sin conexión se transmitan correctamente una vez restablecida la conectividad.
- Los resultados se analizarán y documentarán para determinar la viabilidad técnica y operativa del sistema propuesto.

- El prototipo hará uso de la conexión a internet mediante Wi-Fi para realizar las sincronizaciones con los datos existentes en el servidor como así las funcionalidades de reconocimiento facial avanzado.

1.6 Resumen capítulo

El capítulo 1 presenta el contexto normativo del control de asistencia en Chile y expone las limitaciones de los sistemas biométricos actuales, especialmente su dependencia de internet y su baja interoperabilidad con ERP. Ante esta problemática, se propone un sistema IoT basado en ESP32-CAM con autenticación por rostro y huella, capaz de operar offline y sincronizar datos posteriormente. Además, se establecen los objetivos, alcances y lineamientos generales del proyecto.

2. Marco teórico

A continuación se presentan los fundamentos teóricos y tecnológico que facilitan la comprensión del proyecto

2.1 Conceptos básicos

2.1.1 Conceptos globales

- Control de asistencia: Proceso por el cual una empresa u organización registra las horas de entrada y salida de sus trabajadores.
- Sistema ERP: EntERPrise Resource Planning, sistema de gestión empresarial que permite procesos de una empresa u organización, tales como recursos humanos, contabilidad, finanzas, inventario, asistencia.
- Internet de las cosas (IoT): Paradigma tecnologico que permite la conexión entre dispositivos físico a través de redes de comunicación.
- Resolucion exenta n°38: Norma promulgada por la dirección del trabajo (DT) en 2024 (actualizada en 2025), que establece los requisitos obligatorios para los sistemas electrónicos de registro y control de asistencia.
- Biometria: La biometría corresponde al conjunto de técnicas utilizadas para identificar personas mediante características físicas o conductuales únicas, tales como el rostro o la huella digital. Este tipo de autenticación es ampliamente utilizada en sistemas de control de asistencia por su alta precisión y dificultad de suplantación [36].
- Sincronización offline-online: Proceso mediante el cual un dispositivo opera sin conexión utilizando almacenamiento local y, una vez restablecida la conectividad, transmite los datos acumulados hacia un servidor o ERP. Resulta esencial en entornos con conectividad inestable.

2.1.2 Conceptos de software

- ESP32-CAM: Microcontrolador basado en el esp32, con la inclusión de una cámara OV2640, conectividad WiFi y Bluetooth, además de contar con la capacidad de procesamiento local. [6]
- API (Aplicacion Programming Interface): Conjunto de métodos o puntos de conexión que permiten la conexión entre diferentes aplicaciones mediante protocolos de redes. [7]
- Backend: Parte del sistema encargada de la lógica del negocio, manejo de datos y comunicación con bases de datos. [8]
- Frontend: Conjunto de interfaces y funcionalidades con las que interactúa el usuario final. [9]
- Mqtt: Protocolo de comunicación ligero basado en el modelo publicador/suscriptor, diseñado para intercambiar mensajes de manera eficiente, utilizado entre dispositivos que cuenten con recursos limitados.[10]
- Javascript Object Notation (JSON): Formato de datos que se suele utilizar en entornos de desarrollo web para el intercambio de datos de manera legible tanto por las personas como las maquinas. [11]
- SPIFFS (SPI Flash File System): Sistema de archivos diseñado para microcontroladores que permite almacenar información en memoria flash mediante archivos. En este proyecto se utiliza para guardar usuarios, turnos, asignaciones y asistencias en formato JSON, permitiendo el funcionamiento offline [37].
- API RESTful: Estilo de arquitectura de servicios web basado en el protocolo HTTP, que permite la comunicación entre aplicaciones usando operaciones estándar como GET, POST, PUT o DELETE. Es utilizado para integrar el dispositivo IoT con sistemas ERP externos mediante intercambio de datos en formato JSON [38].
- Microservicios: Arquitectura de software basada en dividir un sistema en servicios independientes que se comunican entre sí. Esta estructura permite escalabilidad y facilita la separación entre el procesamiento de imágenes, backend y comunicación con ERP[39].
- Broker MQTT: Servidor encargado de recibir, gestionar y distribuir mensajes enviados por dispositivos IoT utilizando el protocolo MQTT. Actúa como intermediario entre el publicador (el ESP32-CAM) y los suscriptores (backend u otros servicios)[40].

2.2 Tecnologías utilizadas

A continuación se presentarán todas las tecnologías que harán presente durante el desarrollo del proyecto. Dentro de estas

2.2.1 Microcontrolador ESP32-CAM

El ESP32-CAM es un dispositivo de desarrollo que integra un microcontrolador ESP32-CAM con conectividad Wi-Fi y Bluetooth, además de una cámara OV2640. La elección de este dispositivo se debe a su bajo costo, capacidad de procesamiento local, compatibilidad con el entorno de Arduino y su amplia comunidad de soporte. De las características que se pueden destacar encontramos la captura de imágenes para el reconocimiento facial, gestionar sensores biométricos y mantener conectividad directa con el servidor o la red local. Otras alternativas como la Raspberry Pi Zero W o Arduino MKR1000 fueron descartadas por su mayor consumo y costo implementación. [12]

2.2.2 Lector de huella digital AS608

El lector de huellas digital usa el sensor AS608, un lector de huella con la capacidad de registrar y verificar la identidad de los usuarios mediante patrones dactilares. La elección de este dispositivo fue por la compatibilidad con el ESP32-CAM, su bajo consumo y su resistencia a condiciones de luz ambiental en comparación a otros sensores ópticos tradicionales como el R307. El AS608 se comunica con el microcontrolador a través de una interfaz UART, y la integración de este se realiza mediante la biblioteca Adafruit FingERPint Sensor Library, lo que facilita el registro, búsqueda y verificación de huellas directamente desde el dispositivo IoT. [13]

2.2.3 Protocolos de comunicación HTTP

El protocolo de comunicación HTTP (Hypertext Transfer Protocol) es un protocolo cliente-servidor utilizado para transmitir información mediante solicitudes y respuestas que se transmite a través de los métodos GET, POST, PUT Y DELETE, generalmente utilizando formatos estructurados como JSON (JavaScript Object Notation). Permite la comunicación directa entre dispositivos y servidores, por lo que para este proyecto se le puede dar un buen uso para realizar la petición de información.[14]

2.2.4 Protocolo de comunicación MQTT

El protocolo de comunicación MQTT o Message Queuing Telemetry Transport es un protocolo ligero de mensajería diseñado específicamente para entornos IoT. Se base en la arquitectura de publicador-suscriptor, donde los dispositivos envían mensajes a un broker central que los distribuye entre los clientes suscritos a los distintos tópicos que pueden existir. Este modelo reduce el uso de ancho de banda, lo que lo hace ideal para redes con conectividad limitada o intermitente. [15]

2.2.5 Contenedores Docker

Docker es una plataforma de virtualización ligera que permite empaquetar aplicaciones junto las dependencias dentro de servidores virtuales aislados denominados contenedores. Cada contenedor ejecuta un entorno idéntico al que se utilizaría en producción, lo que facilita el despliegue, la portabilidad de los sistemas. Para el área de IoT y web, Docker es útil para ejecutar servicios como bases de datos, servidores o brokers MQTT de forma independiente y controlada, lo que lo hace útil para el trabajo al momento de levantar los servicios en distintos servidores. [16]

2.2.6 Sistemas de bases de datos

Un sistema de gestor de bases de datos es un conjunto de programas que permiten administrar bases de datos de manera estructurada. La principal función es proporcionar una interfaz entre los usuarios y los datos almacenados, garantizando integridad consistencia y seguridad de la información. Los SGBD relacionales, basado en el modelo propuesto por Edgar F. Codd (1970), organizan los datos en tablas vinculadas mediante relaciones, lo que facilita su consulta a través del lenguaje SQL. Lo SGBD más utilizados que se destacan son MySQL, MariaDb, Oracle Database y PostgreSQL. Estos sistemas ofrecen las características fundamentales, tales como el control de concurrencia, mecanismos de respaldo y recuperación, gestión de usuarios y soporte para transacciones atómicas (ACID). PostgreSQL es un sistema de base de datos relacional y orientado a objetos de código abierto, desarrollado inicialmente en la Universidad de California en Berkeley, caracterizada principalmente por ofrece un motor avanzado con soporte para consultas complejas, procedimientos almacenados, tipos de datos personalizados, índices avanzados y compatibilidad con JSON y XML, lo que permite la fácil integración con aplicaciones web e IoT. [17] Para el contexto del proyecto PostgreSQL se utiliza como base de datos, ya que tiene compatibilidad con entornos de desarrollo como python.

2.3 Estado del arte

A continuación se revisaran los sistemas y tecnológicas existentes relacionados dispositivos de control de asistencia biométricos y su integración con plataformas empresariales, con el propósito de identificar el estado que se encuentra tanto el mercado actual como los avances más relevantes e indicar las limitaciones que motivan el desarrollo de la propuesta de este proyecto.

Si bien los sistemas de control de asistencia es un campo del cual se tienen variados recursos y sistemas, la gran mayoría se encuentran ligados a un software propio, restringiendo a los sistemas ERP tener que pagar suscripciones a estos sistemas.

2.3.1 Soluciones basadas en hardware

En esta sección hará énfasis en aquellos dispositivos de reloj control biométricos que permiten registrar la asistencia de los trabajadores mediante la verificación de características únicas, como huellas digitales, reconocimiento facial, entre otras. Estos equipos incluyen funcionalidad tales como almacenamiento interno, conectividad por red (LAN/Wi-Fi), a continuación se describirán los principales dispositivos utilizados en Chile.

1. GeoVictoria, relojes de control: GeoVictoria dentro de sus planes y productos se encuentran hardware biométrica con una plataforma en la nube para realizar el control de asistencia. Estos dispositivos pueden operar mediante una conexión LAN o 3G para zonas remotas. Sin embargo dentro de sus limitantes encontramos que su API esta sujeto a restricciones comerciales y acceso mediante suscripción, lo que limita la personalización o integración libre. Además se tiene un ecosistema cerrado que dificulta modificaciones independientes por parte del cliente [18].
2. ZKTeco, terminales biométricas de asistencia: ZKTeco es una marca internacional con presencia en Chile y Latinoamérica en general, esta compañía ofrece terminales de control de asistencia "stand-alone" que combinan distintos métodos de registro tales como huella, rostro, PIN y tarjeta. Su limitante es que se encuentra cerrado totalmente a su sistema o quedan sin integración directa con un ERP, a su vez que ofrece exportación de datos mediante archivos csv siendo poco amigable para su uso en un ERP [19].
3. Dispositivos genéricos: Consideramos como dispositivos genéricos cualquier tipo de reloj control que se pueda adquirir en algún portal de venta web, que incluyan características tales como marcaje mediante huella o rostro. En base a lo anterior la gran mayoría de estos dispositivos tienen la capacidad de aplicar estas funciones

pero al costo que su información solo queda en el dispositivo, teniendo métodos como exportacion de datos por usb en un formato que solo se pueden visualizar y difícil de integrar a ERP.

2.3.2 Soluciones basadas en software

En el siguiente apartado se presentarán soluciones relacionadas al control de asistencia relacionados con plataformas web o servicios en la nube (Saas) que permiten gestionar la asistencia laboral sin requerir un reloj biométrico físico.

1. Talana/Buk: Talana y Buk son plataformas chilenas que ofrecen módulos de gestión de asistencia, turnos, entre otras características mediante aplicaciones web, en ambas aplicaciones se encuentran en la misma situación, con dependencia de conectividad a internet, ausencia de hardware biométrico físico, y costos asociados a suscripciones web [20][21].
2. Defontana/Softland: Defontana y Softland son empresas dedicadas a implementar sistemas ERP para las empresas, dentro de sus funcionalidades integran control de asistencia, permitiendo registrar las jornadas de trabajo, cálculo de horas y conexión con módulo de remuneraciones. La gran limitante de estas aplicaciones web que están ligadas 100% a su uso web, es decir, se debe contratar el ERP completo [22][23].

De los productos y servicios presentados anteriormente podemos concluir que el mercado chileno cuenta con una oferta amplia en cuanto a soluciones para el control de asistencia, tanto en dispositivos biométricos, como en plataformas en la nube mediante software. Sin embargo cada uno de los elementos presentados en este estudio del arte, presentan limitaciones en cuanto a su estructura.

Por un lado tenemos los relojes de control biométricos ofrecidos por GeoVictoria o ZKTeco, que tienen soluciones bastante adaptadas a las necesidades del usuario pero con un enfoque en empresas más grandes, ligados a software propietario de una conectividad constante con los servidores del fabricante, esto reduciendo la flexibilidad, elevando costos para los servicios que los ERP finalmente ofreceran a las medias y pequeñas empresas.

Por otra parte las soluciones basadas en software o servicios SaaS, tales como Talana, Buk, Defontana y Softland se centran en la gestión de la asistencia de manera remota y mediante plataformas web, limitándose en su ambiente operacional dedicado solo para trabajar sobre los mismos servicios de la misma empresa, además de haber ausencia de hardware, lo que limita la aplicabilidad en entornos con condiciones poco ideales para el funcionamiento con internet constante.

En consecuencia de lo comentado anteriormente, se evidencia una falta de soluciones híbridas que puedan unir elementos como la autenticación biométrica, con la integración configurable hacia sistemas ERP, sin depender totalmente de una infraestructura externa.

2.4 Metodologías

En este apartado se presentan las metodologías de desarrollo y evaluación que serán utilizadas para este proyecto.

2.4.1 Metodología de desarrollo

Considerando que el proyecto está relacionado con el desarrollo de un prototipo de IoT, y pensando en su futura implementación en un entorno real, se identificaron distintos tipos de metodologías de desarrollo que permitan desarrollar el prototipo de manera que cada iteración, ciclo, o tarea sea una funcionalidad nueva que se adhiera al prototipo. Dado lo anterior se ha realizado una búsqueda para identificar la metodología que más acorde esté para este proyecto. Dentro de las metodologías que si encajaban, dado el contexto del proyecto se encontraron las siguientes metodologías: Scrum, y el modelo de prototipado evolutivo incremental. Dado que Scrum requiere de más actores que estén presentes en el proyecto y al adaptarla para un trabajo en solitario se perderían elementos de esta metodología tales como los roles, como así la revisión colaborativa que son parte de la estructura de la metodología[24]. Es por esta razón se descarto esta metodología y se optó por emplear una metodología de prototipado evolutivo incremental adaptándola a un proyecto en solitario, es decir, se reemplaza lo que sería el cliente, que es quien solicita los requisitos, por el profesor guía quien guiará y entregará retroalimentación de las iteraciones. Esta metodología se adecúa para este proyecto al tratarse de una metodología más apta y preparada para lo que es el desarrollo de prototipos y de mejoras e integraciones continuas[25][26].

La metodología de prototipado evolutivo con un enfoque incremental propone un desarrollo basado en versiones sucesivas del prototipo, en la que cada iteración incorpora nuevas funcionalidades y mejoras sobre el anterior, de esta forma se puede probar, evaluar y ajustar continuamente el sistema a medida que va evolucionando. La estructura de la metodología es la siguiente:

- **Diseño y planificación:** Se definen los objetivos específicos de cada iteración, se seleccionan componentes y se diseña la arquitectura.
- **Implementación y prueba:** Se desarrolla la funcionalidad planificada, se integra con los módulos que ya existen y se verifica su correcto funcionamiento.

- Evaluación y mejora: Una vez desarrollado los procesos anteriores se analizan los resultados obtenidos identificación errores y planificando ajustes para la siguiente iteración.

[27]

Cada una de estas fases termina con un incremento funcional validado, lo que permite mantener un progreso controlado y documentado del prototipo. A continuación se presenta un diagrama en la cual representamos la metodología propuesta.

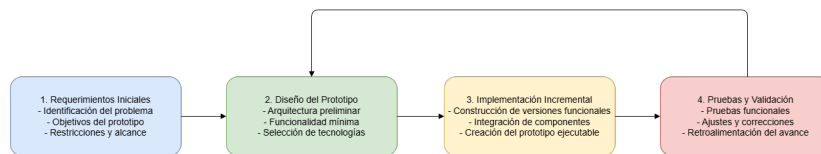


Figura 2.1: Metodología prototipado evolutivo con enfoque incremental

2.4.2 Metodología de evaluación

Para la evaluación del proyecto se deben considerar 3 aspectos claves para determinar la viabilidad del proyecto, de los cuales nos permiten evaluar y verificar nuestros objetivo. Las pruebas evaluarán el prototipo, enfocandose en como funciona con todas las piezas y/o software en conjunto, saber que tan útil puede llegar a ser, y finalmente al tener como objetivo su uso en un ERP, se necesita saber el nivel de satisfacción que se tiene por el prototipo final.

Pruebas de integración

Las pruebas de integración permiten evaluar diferentes componentes de un sistema ya sea hardware software o base de datos, asegurando que todos estos componentes funcionen correctamente en conjunto asegurando la comunicación e interoperabilidad entre ellos. Para este caso se realizarán pruebas de integración ascendentes de manera que se vayan realizando pruebas de los módulos más pequeños hasta llegar al módulo más grande [28]. La selección de esta prueba va relacionada con la metodología de desarrollo que consite en el prototipado evolutivo, el cual se integran progresivamente los módulos validos en etapas previas.

Pruebas de usabilidad

Las pruebas de usabilidad tienen como objetivo evaluar la facilidad de uso, comprensión y satisfacción del usuario con el sistema, conforme a los estándares de calidad de software [29] [30]. Para llevar a cabo las pruebas de usabilidad se utilizará el System Usability Scale (SUS) [31]. El SUS es un método reconocido por su simplicidad de aplicación

y por permitir una evaluación rápida y fiable de la usabilidad de un sistema [32]. La metodología del SUS consiste en obtener una puntuación global de usabilidad mediante la aplicación de un cuestionario basado en una escala de Likert de 5 puntos, compuesto por 10 preguntas que abordan diferentes aspectos de la experiencia de usuario, como la facilidad de uso, la claridad de la interfaz y la eficacia del sistema [31].

La razón del uso de esta prueba se debe a que el prototipo propuesto "prototipo de Control de asistencia Biometrico", debe ser fácil de usar para los trabajadores puedan realizar marcaciones sin complicaciones, como así para quienes integran este sistema a su ERP.

Pruebas de confiabilidad y disponibilidad

Las pruebas de confiabilidad y disponibilidad son cruciales en sistemas que operan en entornos con conectividad intermitente o hardware limitado, como los dispositivos IoT[33]. La finalidad de la prueba de confiabilidad es medir la capacidad del sistema para funcionar correctamente bajo condiciones normales durante un periodo prolongado sin errores ni pérdidas de datos, mientras que las pruebas de disponibilidad hace referencia a la capacidad del sistema para mantenerse operativo incluso cuando se presentan condiciones extremas, tales como pérdidas conexión, desconexión temporal de energía[34], etc. P Según Perry & Petry, estas pruebas son esenciales para garantizar que el sistema mantenga la integridad de los datos y su disponibilidad en todo momento[35].

2.5 Resumen del capítulo

El capítulo 2 presenta el marco teórico que sustenta el proyecto, describiendo los conceptos esenciales relacionados con control de asistencia, IoT, ERP, y los requisitos normativos definidos por la Resolución Exenta N.º38. Luego se detallan las tecnologías utilizadas, incluyendo el microcontrolador ESP32-CAM, el lector de huellas AS608, los protocolos HTTP y MQTT, el uso de contenedores Docker y la base de datos PostgreSQL.

También se revisa el estado del arte, analizando las limitaciones de dispositivos biométricos comerciales y plataformas SaaS como Talana, Buk, Defontana y Softland, evidenciando la falta de soluciones híbridas, abiertas y con funcionamiento offline. Finalmente, el capítulo expone las metodologías de desarrollo y evaluación seleccionadas, donde se justifica el uso del prototipado evolutivo incremental y se describen las pruebas de integración, usabilidad y confiabilidad que se aplicarán al sistema.

3. Metodologías

En el presente capítulo esta destinado a describir la manera en la que se aplicarán las metodologías propuestas en el capítulo anterior, además de presentar los resultados esperados.

3.1 Metodologia de desarrollo

Como se menciona el capítulo anterior en la sección 2.4.1 el proyecto propuesto se lleva a cabo mediante una metodología de tipo prototipado evolutivo con enfoque incremental. Esta metodología permite crear versiones funcionales del prototipo de manera progresiva, mejorando en cada iteración en base a pruebas funcionales del prototipo.

A continuación se procederá a explicar las distintas fases del proceso y como se aplicará la metodología.

3.1.1 Protipado Evolutivo Incremental

La metodología de prototipado evolutivo incremental se aplica en este proyecto como un enfoque de desarrollo iterativo cuyo objetivo principal es construir el prototipo del sistema de manera progresiva, mediante versiones funcionales que incorporan nuevas capacidades en cada iteración. A diferencia de modelos secuenciales como el modelo en cascada, este enfoque permite validar tempranamente el funcionamiento real del dispositivo IoT, ajustar sus componentes según las pruebas realizadas y asegurar que cada incremento aporte mejoras concretas sobre el prototipo anterior.

Esta metodología se adecuía debido a que la propuesta de proyecto combina hardware (ESP32-CAM, lector de huella AS608 y sensor IR), software embebido, almacenamiento local, backend, mecanismos biométricos y una interfaz web. Dado que estos elementos funcionan como módulos independientes que deben integrarse correctamente, un modelo basado en incrementos funcionales permite validar cada módulo antes de avanzar al siguiente nivel de complejidad.

El proceso de prototipado evolutivo incremental se aplica siguiendo tres fases principales: diseño y planificación, implementación y prueba, y evaluación y mejora, las cuales se repiten en cada iteración del proyecto. A continuación, se explica cómo se aplican estas fases dentro del desarrollo del prototipo.

1. Fase de diseño y planificación

En la fase de diseño y planificación se definen los objetivos específicos de la iteración correspondiente, así como los módulos que se incorporarán al prototipo. Para cada ciclo se identifican los componentes de hardware y software involucrados, los flujos de datos necesarios y la arquitectura parcial que debe soportar la nueva funcionalidad.

Durante esta fase se desarrollan diagramas preliminares, modelos de datos, mockups y definiciones de comunicación entre módulos. Asimismo, se determinan las dependencias entre componentes, lo que permite asegurar que el incremento se pueda integrar adecuadamente con los elementos ya existentes. Esta etapa garantiza que cada iteración tenga un alcance claro, siguiendo un orden lógico de complejidad creciente.

2. Fase de implementación y prueba

En esta fase se procede a construir la funcionalidad definida en el diseño de la iteración. La implementación incluye el desarrollo del firmware del ESP32-CAM, la programación del lector de huellas AS608, la gestión del almacenamiento en SPIFFS, el desarrollo del backend o la implementación de los modelos biométricos, según corresponda al incremento.

Paralelamente, se realizan pruebas integradas para verificar que la nueva funcionalidad opere correctamente en conjunto con las versiones anteriores del prototipo. Estas pruebas incluyen verificaciones de comunicación entre módulos, consistencia de datos, estabilidad del firmware, validación de imágenes capturadas, o evaluación de la sincronización con el servidor.

El objetivo es garantizar que el incremento sea plenamente funcional antes de ser incorporado como parte estable del prototipo.

3. Fase de evaluación y mejora

Finalizada la implementación, cada incremento es sometido a una fase de evaluación donde se analizan los resultados obtenidos. En esta etapa se identifican errores, limitaciones y posibles mejoras, considerando el funcionamiento real del dispositivo tanto en escenarios conectados como offline.

Cuando corresponde, se realizan ajustes al diseño, se corrigen fallas detectadas y se optimiza el desempeño de los componentes integrados. Esta fase también permite

preparar la entrada para la siguiente iteración, asegurando que el prototipo evolucione de manera controlada y mantenga coherencia con los objetivos del proyecto.

3.2 Planificación

La planificación del proyecto se estructura mediante la metodología de prototipado evolutivo con enfoque incremental. Cada iteración incorpora nuevas funcionalidades sobre el prototipo anterior, permitiendo evaluar, corregir y fortalecer el sistema progresivamente. A continuación, se detalla cada iteración, incluyendo sus fases de análisis, diseño, implementación y pruebas.

Con el fin de poder contrarrestar el avance del desarrollo del proyecto cada iteración corresponde a un 12,5% del total del proyecto, donde además cada porcentaje de tarea corresponde a la división de 12,5% entre la cantidad de tareas a realizar.

3.2.1 Iteración 1: Diseño y configuración inicial

Análisis

- Identificación de requisitos base: captura facial, lectura de huella y almacenamiento.
- Definición de restricciones del proyecto: memoria del ESP32, calidad de imagen y conectividad Wi-Fi.
- Selección preliminar de arquitectura del sistema IoT.

Diseño

- Diseño de la arquitectura general del sistema.
- Diseño de mockups base para la interfaz del dispositivo.
- Diseño del flujo operacional inicial.

Implementación

- Configuración inicial del ESP32-CAM.
- Programación del lector AS608 mediante UART.
- Integración funcional del lector de huellas con el ESP32-CAM.
- Implementación de mockups planteados en la etapa de diseño.

Pruebas

- Prueba de lectura de huella.
- Prueba de captura de imagen desde el ESP32-CAM.

3.2.2 Iteración 2: Almacenamiento local y modo offline

Análisis

- Selección del formato de almacenamiento (archivos JSON mediante SPIFFS).
- Identificación de parámetros configurables por el usuario (Wi-Fi, endpoint, datos locales).

Diseño

- Diseño de estructura JSON para usuarios, turnos, asignaciones y asistencias.

Implementación

- Implementación del almacenamiento offline mediante SPIFFS.
- Implementación de marcacion offline mediante huella dactilar

Pruebas

- Verificación de persistencia tras reinicios.
- Prueba de lectura y escritura de archivos locales.

3.2.3 Iteración 3: Comunicación y conectividad

Análisis

- Evaluación del uso de HTTP y MQTT como protocolos principales de envío de datos.
- Análisis de la conectividad vía Wi-Fi para operación online y offline.

Diseño

- Diseño de Diagrama de Bases de Datos.
- Diseño de la API backend para recepción de imágenes y registros.
- Diseño de endpoints para sincronización e integración con la base de datos.

Implementación

- Implementación de la base de datos en el backend.
- Implementación del envío de imágenes desde el ESP32 al backend.
- Implementación del envío de registros biométricos.
- Programación de reintentos de conexión.

Pruebas

- Prueba de recepción de datos en backend.
- Prueba de latencia y estabilidad de transmisión.

3.2.4 Iteración 4: Detección de rostro y validación biométrica**Análisis**

- Evaluación de librerías de detección de rostros.
- Revisión de compatibilidad con imágenes provenientes del ESP32-CAM.

Diseño

- Diseño del flujo captura–detección–validación.
- Definición del proceso de comparación entre rostro y huella almacenados.

Implementación

- Implementación de detección de rostros en backend.
- Validación biométrica combinada (rostro + huella).
- Registro automático en la base de datos.

Pruebas

- Pruebas de detección bajo distintas condiciones de luz y distancia.
- Verificación del funcionamiento de validación biométrica.

3.2.5 Iteración 5: Módulo antifraude mediante sensor IR

Análisis

- Identificación de escenarios de suplantación (fotos, pantallas).

Diseño

- Diseño de la lógica antifraude basada en detección física.
- Integración de lectura IR al flujo de captura facial.

Implementación

- Integración del sensor IR al ESP32-CAM.
- Programación de validación previa a la captura de rostro.

Pruebas

- Pruebas con fotografías, pantallas y objetos.
- Pruebas en distintas distancias e iluminación.

3.2.6 Iteración 6: Página web backend e integración con ERP

Análisis

- Definición del público objetivo: administradores, trabajadores.
- Definición de qué datos deben visualizarse (asistencias, imágenes, usuarios, estado del dispositivo).

Diseño

- Diseño del panel web backend con vistas estructuradas.
- Diseño del módulo para enviar datos a sistemas ERP externos.

Implementación

- Implementación del panel web de administración.
- Integración del módulo de exportación a ERP mediante endpoint configurable.

Pruebas

- Pruebas de visualización y navegación.
- Pruebas de integración con ERP simulado.

3.2.7 Iteración 7: Validación final del prototipo**Análisis**

- Definición de métricas de desempeño: confiabilidad, disponibilidad, latencia.
- Análisis de escenarios reales de uso.

Diseño

- Diseño de pruebas de estrés, disponibilidad y reconexión.
- Diseño del proceso de evaluación de usabilidad (SUS).

Implementación

- Ejecución del prototipo en entornos simulados y reales.
- Aplicación de encuestas SUS a usuarios de prueba.

Pruebas

- Pruebas de desempeño y disponibilidad.
- Verificación de integridad de datos en sincronización offline-online.

3.2.8 Iteración 8: Integración total y cierre del proyecto**Análisis**

- Revisión del cumplimiento de objetivos y requisitos normativos.
- Identificación de mejoras finales necesarias.

Diseño

- Diseño del paquete final del prototipo (firmware, backend, panel web y manuales).

Implementación

- Integración final de todos los módulos del sistema.
- Elaboración de manuales técnicos e instructivos.

Pruebas

- Validación completa del sistema en flujo de punta a punta.
- Revisión final con profesor guía.

3.3 Metodología de evaluación

En esta sección se explica de qué manera se aplicaron, dentro del desarrollo del prototipo, las metodologías de evaluación descritas en el apartado 2.4.2 del marco teórico. Cada técnica de evaluación fue seleccionada en función de los objetivos del proyecto y del enfoque metodológico utilizado prototipado evolutivo con incrementos funcionales, lo que permitió validar el funcionamiento conjunto del hardware, software, backend y mecanismos de autenticación biométrica en entornos reales o simulados. Las evaluaciones se aplicaron de manera iterativa al finalizar cada incremento del prototipo, garantizando que las funcionalidades incorporadas funcionaran correctamente antes de avanzar al siguiente nivel de complejidad.

3.3.1 Pruebas de integración

Las pruebas de integración, como por su nombre dice son pruebas que permiten integrar componentes de un sistema verificando que su interaccion sea correcta cuando se implementa un nuevo modulo o dispositivo como es en el contexto de este proyecto. Dado que el proyecto esta compuesto por multiples módulos independientes, lector de huellas, camara del ESP32-CAM, el almacenamiento local, backend, reconocimiento facial, anti fraude IR, página web y modulo ERP, es necesario verificar su interoperabilidad a medida que se ensamblan.

Con el objetivo de asegurar que todos los componentes mencionados anteriormente interactuen correctamente entre si se realizaran pruebas de integración de manera ascendente. El procedimiento que irá realizando el siguiente procedimiento.

1. integración del hardware: Se evaluara la comunicación entre el ESP32-CAM y el modulo de huella digital, Además de comprobar la lectura y envio de datos de sensores IR.

2. integración Almacenamiento: Se realizaran pruebas entorno al almacenamiento de los identificadores de huella, imágenes capturadas además de los registro de asistencia, donde se verificara la consistencia del almacenamiento.
3. integración con el backend: Se enviaran distintos tipos de datos al backend, imágenes en base64, registros de logs, entre otros para verificar la comunicación entre el ESP32 y el backend.
4. integración Backend con la base de datos: Se realizaran inserciones y búsqueda de elementos para verificar la conectividad constante con la base de datos.
5. integración backend con interfaz web: Se comprobara si se rescatan los registros enviados por el prototipo y actualización en tiempo real con los datos.

Estas pruebas permiten evaluar el sistema propuesto de manera sistemática, de manera que cada elemento más pequeño se va integrando a medida que se avanza en el proyecto.

3.3.2 Pruebas de usabilidad

Las pruebas de usabilidad SUS como se menciona anteriormente en el punto 2.4.2 permite evaluar la facilidad de uso, la eficacia del sistema o la claridad de la interfaz. Dado el contexto del proyecto de un prototipo de control de asistencia su usuarios esperados serian trabajadores y desarrolladores, es por esto que la prueba de usabilidad y facilidad con la que interactuan usuarios no técnicos como trabajadores, empleadores, y junto a esto la facilidad de integración del sistema destinada para programadores o personal técnico que deberá integrar el prototipo con el ERP. Considerando lo anterior el procedimiento sera de realizar una encuesta con 10 preguntas, donde se responderan en base a la escala Likert. A continuacion se presentarán las preguntas tanto para usuarios no técnicos y para desarrolladores.

Preguntas para usuarios no técnicos

- ¿Qué tan fácil fue encontrar la opción para configurar la red WiFi dentro de la interfaz embebida?
- ¿Lograste modificar los parámetros del dispositivo (como el endpoint del backend) sin necesitar asistencia?
- ¿Te resultó clara la estructura del panel web al momento de buscar registros de asistencia?

- ¿Pudiste identificar rápidamente las imágenes asociadas a cada marcación en el panel web?
- ¿Encontraste intuitivo el uso de los filtros por fecha, usuario o tipo de autenticación?
- ¿La interfaz te entregaba suficiente información para entender si el dispositivo estaba funcionando online u offline?
- ¿En algún momento sentiste confusión respecto a qué botón o sección debías utilizar para continuar una tarea?
- ¿Te pareció clara y legible la información mostrada en la tabla de registros?
- ¿Consideras que podrías aprender a usar la interfaz completa (embebida + panel web) sin instrucción adicional?
- ¿Experimentaste dificultades al intentar visualizar o interpretar algún dato dentro de la interfaz?

Preguntas para usuarios técnicos

- ¿La documentación técnica del backend contenía suficiente información para comenzar a integrar la API sin asistencia externa?
- ¿Pudiste consumir los endpoints del prototipo utilizando herramientas como Postman, cURL o aplicaciones propias sin complicaciones?
- ¿La estructura JSON entregada por el dispositivo te pareció clara, consistente y fácil de mapear a otros sistemas?
- ¿Lograste identificar rápidamente los campos esenciales del registro (timestamp, usuario, métodos biométricos, estado de sincronización)?
- ¿Te resultó sencillo modificar el endpoint configurado en el dispositivo para redirigir los datos hacia tu propio servidor?
- ¿Los mensajes de error entregados por el backend fueron comprensibles y útiles para depurar problemas de integración?
- ¿Encontraste coherencia entre los distintos endpoints del backend (nombres, métodos HTTP, estructura de respuesta)?
- ¿Fue simple integrar o reutilizar las imágenes enviadas por la ESP32-CAM para procesamiento posteriores?

- ¿Consideras que la API del prototipo podría integrarse fácilmente a un sistema ERP real sin necesidad de modificaciones mayores?
- ¿Tuviste dificultades técnicas significativas durante el proceso de integración que consideres relevantes para mejorar el sistema?

3.3.3 Pruebas de confiabilidad y disponibilidad

Para un prototipo IoT con funcionamiento offline, la confiabilidad del almacenamiento local y la disponibilidad del sistema durante fallas de red son esenciales. Estas pruebas evaluarán que tan robusto puede llegar a ser el prototipo frente a escenarios reales de operaciones y fallos. Para el procedimiento se realizará lo siguiente.

- Escenario de conectividad: Para realizar esta prueba se consideran distintos escenarios que permita evaluar el funcionamiento del prototipo en estos escenarios de los cuales consideramos escenarios de desconexión total, conectividad intermitente, reconexión luego de una desconexión prolongada.
- Métricas a evaluar: Dentro de las métricas que se evaluarán corresponden a los porcentajes de registros almacenados sin errores, cantidad de re intentos automáticos, consistencia del log de eventos, tiempo de recuperación tras re conectar (sincronizar datos).
- Verificación con base de datos: Con la finalidad de comprobar la confiabilidad del sistema se debe comparar el número de registros capturados offline en comparación con los almacenados en la base de datos o que fueron sincronizados anteriormente.

3.4 Resultados Esperados

A continuación en esta sección se presentarán los resultados esperados para los objetivos específicos.

- Objetivo específico 2: Implementar el módulo biométrico de registro de asistencia mediante reconocimiento facial y huella digital.
 - Resultado esperado: Para este objetivo se espera tener un prototipo que permita registrar asistencia de un trabajador, mediante reconocimiento facial o huella digital.
- Objetivo específico 3: Incorporar capacidad de funcionamiento offline con almacenamiento local y sincronización posterior.

- Resultado esperado: Para este objetivo se espera que el prototipo pueda tener la capacidad de funcionamiento offline almacenando los registros de manera local para su posterior sincronización con el servidor.
- Objetivo específico 6: Integrar y validar el funcionamiento conjunto del hardware y software desarrollado.
 - Resultado esperado: Para este objetivo se espera que el hardware construido funcione correctamente en conjunto como así su operabilidad con el backend y software.
- Objetivo específico 7: Evaluar el desempeño, confiabilidad y usabilidad del sistema en escenarios de prueba reales.
 - Resultado esperado: Para este objetivo se espera obtener alta disponibilidad con al menos un 90% de su funcionamiento en los escenarios en donde el dispositivo se encuentre offline, o con conexión intermitente.

3.5 Resumen del capítulo

El capítulo 3 describe cómo se aplica la metodología de prototipado evolutivo incremental para el desarrollo del sistema. Se detallan las iteraciones planificadas, cada una incorporando nuevas funcionalidades: configuración inicial del hardware, almacenamiento local y modo offline, comunicación con backend, reconocimiento facial, módulo antifraude con sensor IR, implementación del panel web, integración con ERP y validación final del prototipo.

Asimismo, se explica la metodología de evaluación, que incluye pruebas de integración, usabilidad mediante el método SUS, y pruebas de confiabilidad y disponibilidad para medir el desempeño del sistema en escenarios reales. El capítulo concluye presentando los resultados esperados asociados a los objetivos específicos del proyecto.

4. Desarrollo

En el siguiente capítulo se enfocará en presentar la implementación realizada en base a la metodología de desarrollo.

4.1 Diseño de software

Esta sección está destinada a presentar las arquitecturas desarrolladas para este proyecto como así diagramas y mockups que permitan dar entender la lógica del proyecto como así futuras vistas.

4.1.1 Arquitectura física

La arquitectura física propuesta se basa en una arquitectura basada en microservicios. En la figura 4.1 se puede presenciar como sería la comunicación entre el dispositivo IoT que emite el evento para realizar el registro, para luego que el servidor con el broker MQTT reciba las imágenes y utilice el servicio de reconocimiento facial, para luego enviarlas al servidor backend en donde se distribuyen para el sistema ERP y para la registrarlo en la base de datos, además de disponer del panel web para la visualización de los datos de manera remota.

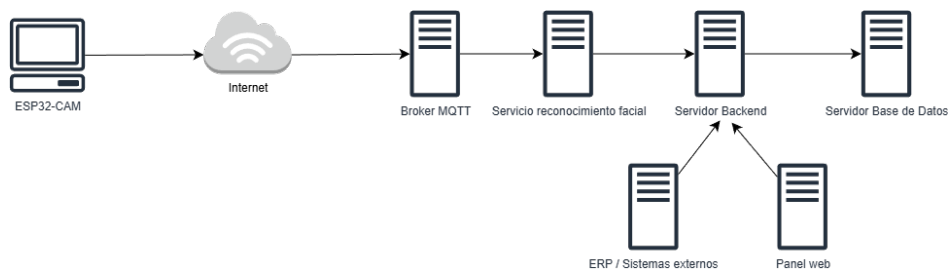


Figura 4.1: Arquitectura física

4.1.2 Arquitectura lógica

La arquitectura lógica del sistema organiza los módulos de software que permiten el funcionamiento del dispositivo IoT y su comunicación con el backend. Tal como se muestra en la Figura 4.2, el ESP32-CAM integra los componentes encargados de la captura de imágenes, la lectura de la huella digital, el almacenamiento local mediante SPIFFS y el servidor web embebido, los cuales interactúan entre sí para registrar usuarios, capturar datos biométricos y almacenar o enviar asistencias.

El backend recibe la información enviada por el dispositivo, procesa las imágenes, administra la base de datos y expone una API que permite la integración con sistemas externos como ERP. Finalmente, la arquitectura lógica contempla la comunicación entre cada módulo mediante protocolos HTTP o MQTT, lo que posibilita el funcionamiento offline, la sincronización posterior y la consulta remota de los datos desde el panel web.

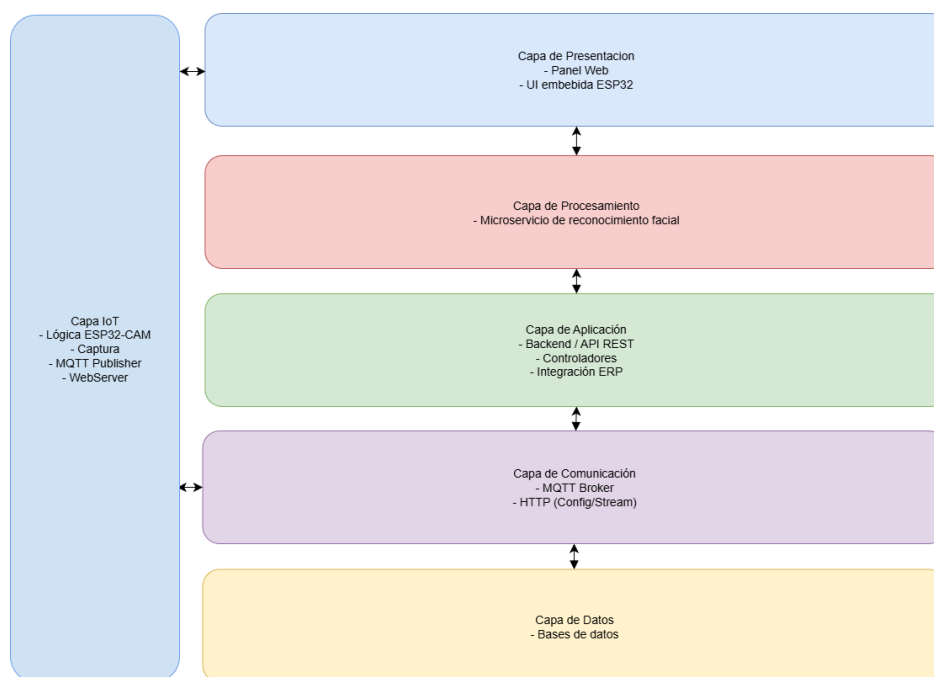


Figura 4.2: Arquitectura lógica

4.2 Implementación

Es este apartado se procederá a explicar como se llevaron a cabo las iteraciones planteadas en el capítulo 3.1.

4.3 Iteración 1: Diseño y planificación inicial

La primera iteración corresponde al diseño conceptual del sistema y a la preparación del entorno de desarrollo, incluyendo la integración inicial del hardware biométrico seleccionado. Esta etapa establece la base funcional sobre la cual se construirán las iteraciones posteriores.

4.3.1 Análisis

La primera iteración se orientó a comprender el alcance inicial del prototipo y a establecer los requisitos mínimos necesarios para validar la viabilidad técnica del sistema. En esta etapa se analizaron los componentes esenciales, las restricciones del hardware y las funcionalidades críticas que debían ser implementadas para permitir la construcción de las siguientes iteraciones.

El análisis se centró en tres ejes principales:

- **Definición de funcionalidades mínimas (MVP):** Se identificaron las capacidades básicas que el dispositivo debía cumplir desde el inicio: capturar imágenes desde la cámara integrada, leer huellas dactilares, entregar páginas HTML embebidas y operar de forma autónoma mediante la red Wi-Fi. Estas funciones representan la base operativa sobre la cual se desarrollarán características más avanzadas como turnos, sincronización y gestión de usuarios.
- **Identificación de limitaciones técnicas:** Se revisaron las restricciones tecnológicas del ESP32-CAM, como la limitada memoria RAM disponible, el uso compartido del bus de cámara, la ausencia de pines GPIO libres y la necesidad de un manejo eficiente de cadenas HTML grandes en memoria. Estas limitaciones influyeron directamente en el diseño de la arquitectura lógica y las decisiones técnicas adoptadas.
- **Estrategia de operación local:** Dado que uno de los requisitos fundamentales del sistema es la autonomía, se analizó la necesidad de montar un servidor web local accesible en mediante la red WiFi. Esto permitiría registrar usuarios, visualizar datos y operar el sistema sin dependencia de internet, reduciendo riesgos y asegurando la continuidad operacional del dispositivo.

Como resultado del análisis, se establecieron los criterios de diseño que guiarían el desarrollo de esta iteración: simplicidad, bajo consumo de recursos, modularidad de componentes y autonomía operativa. Estos lineamientos permitieron definir con claridad las funcionalidades que debían implementarse en la siguiente fase de diseño e implementación.

4.3.2 Diseño

En esta iteración se elaboraron los primeros elementos arquitectónicos del sistema:

- **Arquitectura física**, representando los componentes utilizados y su interconexión, la cual se puede revisar en la Figura 4.1 del 4.1.1 Arquitectura física.
- **Arquitectura lógica**, definiendo los módulos internos del firmware tales como captura de imágenes, lectura de huella y conectividad, la cual se puede revisar en la Figura 4.2 del 4.1.2 Arquitectura lógica.
- **Mockups iniciales** de la interfaz embebida del ESP32-CAM, incluyendo:
 - Menú principal.

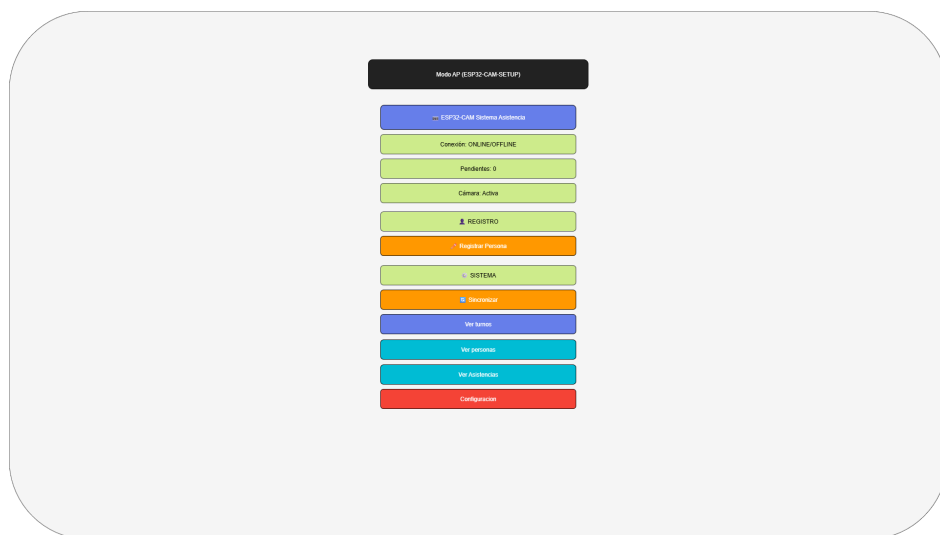


Figura 4.3: Mockup menú principal en el ESP32-CAM

- Panel de configuración de Wi-Fi.

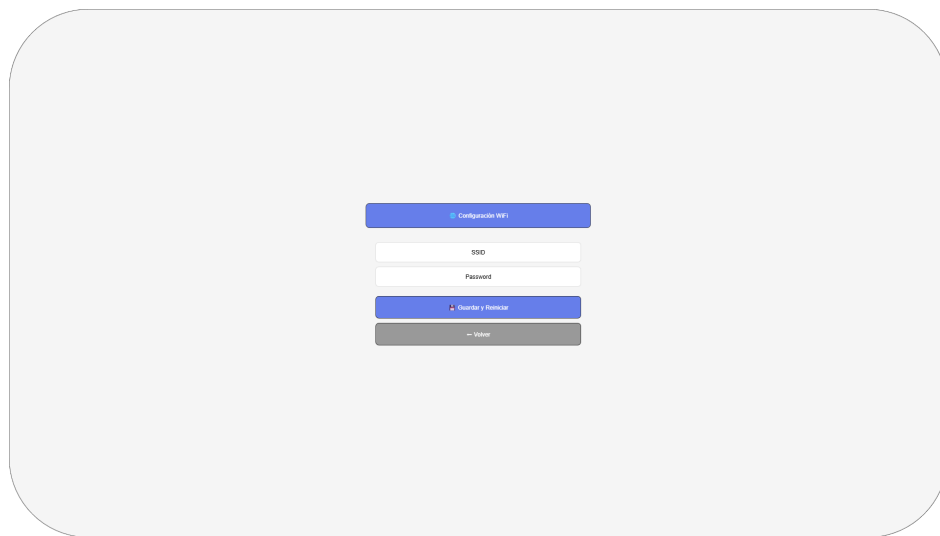


Figura 4.4: Mockup panel de configuración Wi-Fi

– Vista de registro de usuario.

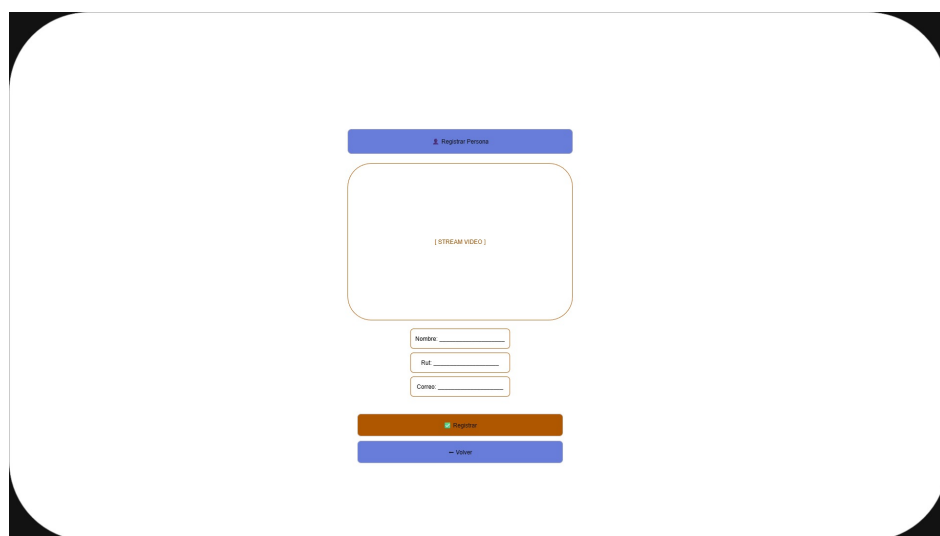


Figura 4.5: Mockup registro usuario en el ESP32-CAM

– Vista de usuarios almacenados localmente.

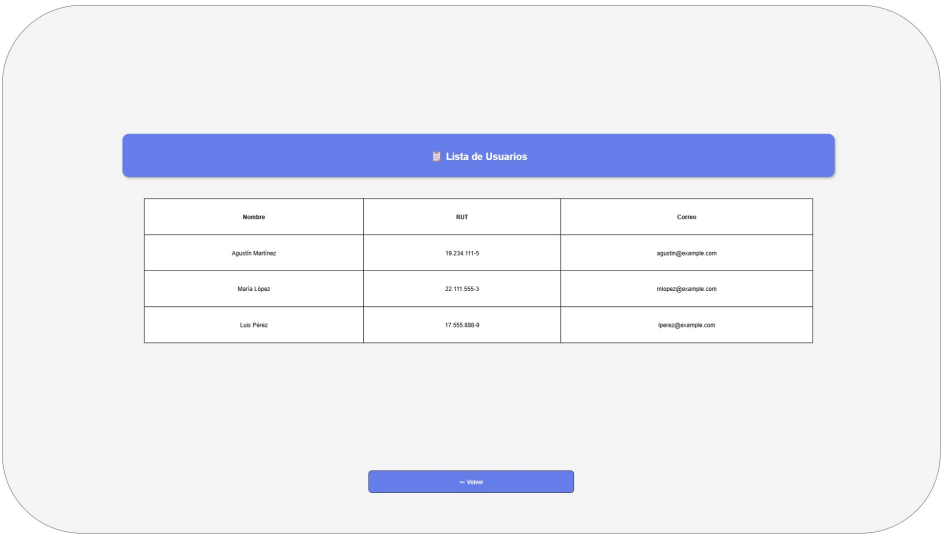


Figura 4.6: Mockup usuarios almacenados en el ESP32-CAM

– Vista de turnos almacenados localmente.

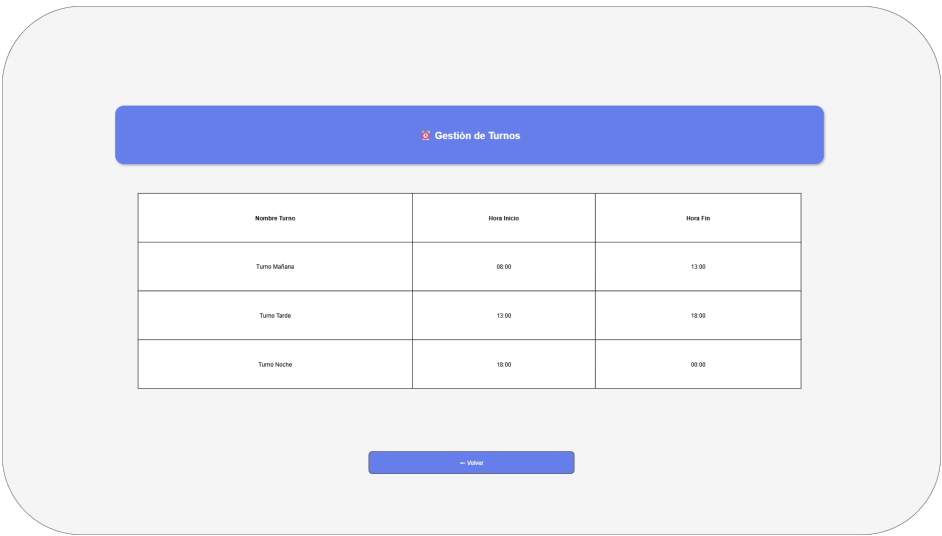


Figura 4.7: Mockup usuarios almacenados en el ESP32-CAM

– Vista de crear turnos.

The mockup for creating shifts is displayed on a light gray background. It features a blue header bar labeled "Gestionar turnos". Below this, there is a form with three input fields: "Nombre del turno:" followed by a wide text box, "Hora Inicio:" with a time picker showing "08:00", and "Hora Fin:" with a time picker showing "13:00". A second blue header bar labeled "Turnos asignados" is positioned below the form. Underneath, there is a white box labeled "Lista turnos creados". At the bottom, there are two buttons: a green "Crear Turno" button and a red "Cancelar" button.

Figura 4.8: Mockup creación de turnos en el ESP32-CAM

– Vista de Asignacion de turnos.

The mockup for assigning shifts is displayed on a light gray background. It features a blue header bar labeled "Asignar Turno". Below this, there are two dropdown menus: "Seleccionar Usuario" with a user icon and "Seleccionar Turno" with a calendar icon. A green "Guardar Asignación" button is located below the dropdowns. A second blue header bar labeled "Asignaciones actuales" is positioned below the button. Underneath, there is a white box labeled "Lista de asignaciones actuales". At the bottom, there is a red "Volver" button.

Figura 4.9: Mockup asignacion de turnos en el ESP32-CAM

- **Diseño del flujo operacional:** Representa el flujo del proceso de marcación para los casos en se cuente con conexión a internet y sin internet.

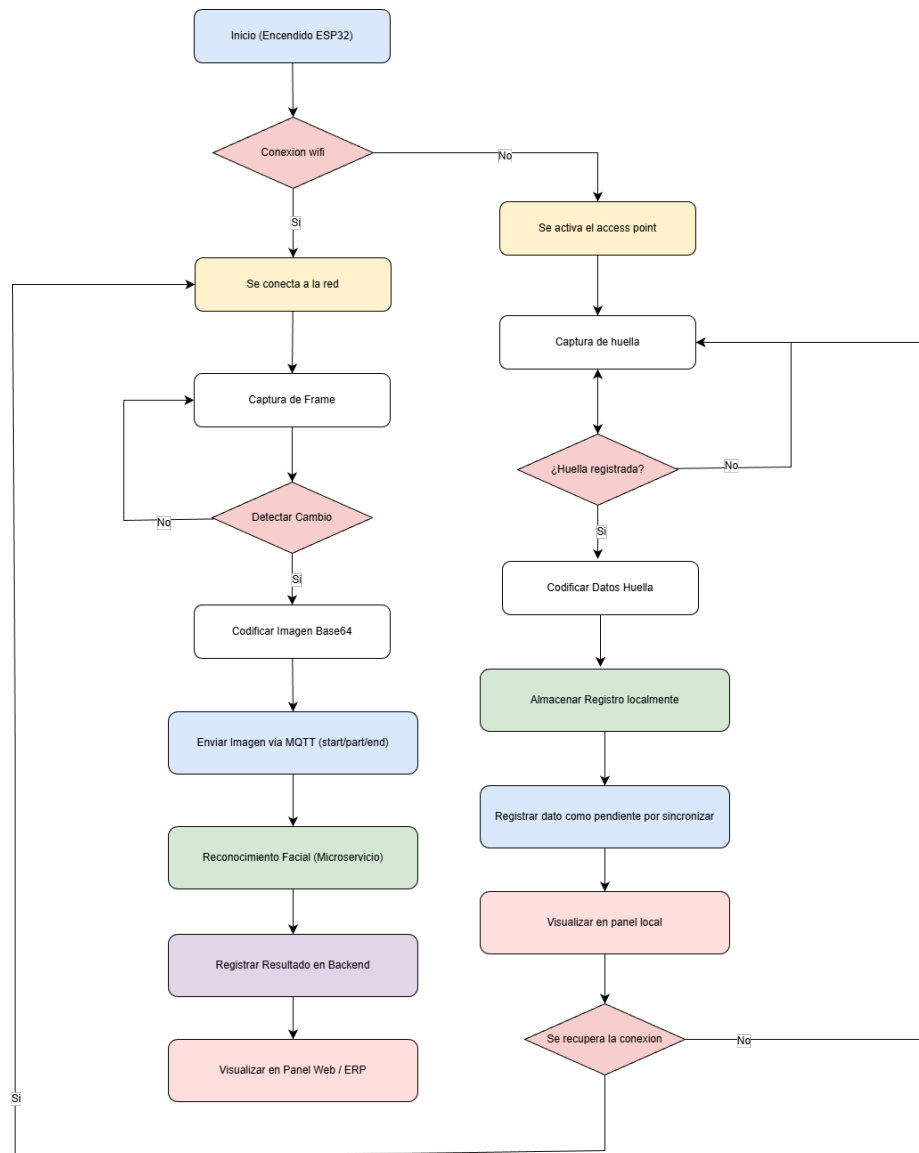


Figura 4.10: Flujo operacional para el registro de asistencia.

Estos elementos permiten visualizar el flujo básico del dispositivo antes de incorporar funcionalidades adicionales como turnos, sincronización o panel web backend, los cuales se desarrollarán en iteraciones posteriores.

4.3.3 Implementación

En esta etapa se integró tanto el hardware principal del prototipo como el desarrollo de las vistas web embebidas que permiten la interacción del usuario con el dispositivo.

Para ello se utilizaron las siguientes librerías:

- `esp_camera`, utilizada para inicializar y configurar la cámara OV2640 integrada en el ESP32-CAM.
- `HardwareSerial` y `Adafruit_Fingerprint`, empleadas para la comunicación UART con el lector biométrico AS608.
- `WebServer`, utilizada para montar un servidor web HTTP y atender rutas asociadas a las vistas.
- `SPIFFS` y `ArduinoJson`, empleadas para almacenamiento persistente en memoria flash y manipulación estructurada de datos JSON.

Integración del lector de huellas AS608

El lector biométrico AS608 fue conectado directamente al ESP32-CAM mediante la interfaz UART secundaria, utilizando los pines GPIO12 (RX) y GPIO13 (TX). Esta conexión permite enviar y recibir comandos tales como enrolamiento, captura e identificación de huellas dactilares. La Figura 4.11 presenta la conexión física entre ambos dispositivos.

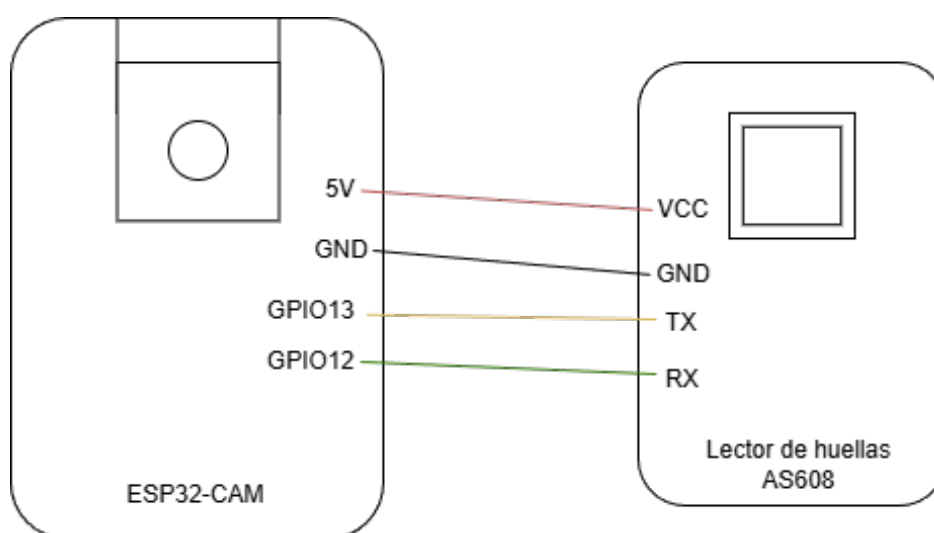


Figura 4.11: Conexiones entre el ESP32-CAM y el lector de huellas AS608.

El módulo se inicializa estableciendo la velocidad de comunicación a 57600 baudios y vinculando el puerto serial con la librería `Adafruit_Fingerprint`. Una vez establecida la comunicación, el firmware puede ejecutar operaciones como:

- Captura de imagen dactilar.
- Conversión de la imagen en una plantilla biométrica.
- Almacenamiento del template en un slot interno del sensor.
- Identificación mediante búsqueda de coincidencias en memoria.

El fragmento de código que se ve en la figura 4.12 es un ejemplo que se contruyo para ser utilizado en iteraciones posteriores para el uso del lectores de huellas junto al ESP32-CAM

```
HardwareSerial FingerSerial(2);
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&FingerSerial);

void initFinger() {
    FingerSerial.begin(57600, SERIAL_8N1, 12, 13);
    finger.begin(57600);

    if (finger.verifyPassword()) {
        Serial.println("Sensor AS608 listo.");
    } else {
        Serial.println("Error: no se detecta el AS608");
    }
}
```

Figura 4.12: Fragmento de código para inicializar el lector AS608 utilizando UART.

Montaje del servidor web embebido

Para habilitar la interacción local con el dispositivo, se implementó un servidor web mediante la librería **WebServer**. Este servidor permite entregar páginas HTML, datos JSON y recursos multimedia como imágenes JPEG o streams MJPEG provenientes de la cámara.

Las rutas principales del sistema incluyen:

- `/`: Menú principal.
- `/wifi`: Configuración de red Wi-Fi.
- `/register`: Registro de personas.
- `/asistencias`: Lista de asistencias almacenadas localmente.
- `/gestion`: Gestionar Turnos

- `/personas`: Visualización personas registradas.
- `/asignaciones`: Vista para realizar asignaciones.

Para la implementación del servidor HTTP embebido, se utilizó el módulo `WebServer` del ESP32. Cada operación del sistema (registro de usuarios, captura de huellas, creación de turnos, asignación de turnos, lectura de asistencias, configuración Wi-Fi, entre otras) fue mapeada a un endpoint específico mediante funciones manejadoras. Este mecanismo permite encapsular la lógica de cada componente del sistema y mantener una estructura modular del firmware.

```
server.on("/", []() { server.send(200, "text/html", htmlPage); });
server.on("/register", []() { server.send(200, "text/html", registerPage); });
server.on("/gestion", []() { server.send(200, "text/html", gestionPage); });
server.on("/personas", []() { server.send(200, "text/html", personasPage); });
server.on("/asistencias", []() { server.send(200, "text/html", asistenciasPage); });
server.on("/turnos", []() { server.send(200, "text/html", turnosPage); });
server.on("/asignaciones-view", []() { server.send(200, "text/html", asignacionesPage); });
server.on("/wifi-setup", []() { server.send(200, "text/html", wifiConfigPage); });
server.on("/wifi-config", handleWiFiConfig);
server.on("/registrar", handleRegisterUser);
server.on("/crear_turno", handleCreateTurn);
server.on("/asignar", handleAssignTurn);
server.on("/marcar", handleMarcarAsistencia);
server.on("/api/personas", handleGetPersonas);
server.on("/api/turnos", handleGetTurnos);
server.on("/api/asignaciones", handleGetAsignaciones);
server.on("/api/asistencias", handleGetAsistencias);
server.on("/limpiar", handleLimpiarDatos);

server.begin();
```

Figura 4.13: asignación de rutas para el servidor web embebido.

Creación de vistas HTML embebidas

Las vistas fueron desarrolladas directamente en el firmware utilizando cadenas literales de gran tamaño, permitiendo que el ESP32-CAM sirva cada página incluso en modo Access Point sin conexión a internet. Esto garantiza autonomía operativa, especialmente durante el registro, la captura y la visualización de asistencias. Cada vista se definió como una constante `R"rawliteral(...)"`, conteniendo HTML, CSS y JavaScript.

Menú principal

La Figura 4.14 presenta la vista principal accesible en la ruta `/`. Desde esta interfaz el usuario puede visualizar el estado del dispositivo, registrar personas, ver asistencias, gestionar turnos, ver asignaciones y acceder a la configuración de red, además de limpiar datos del dispositivo.

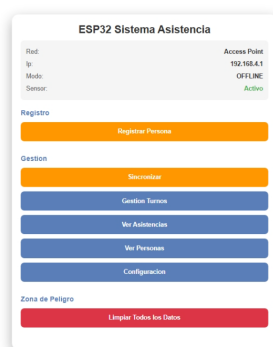


Figura 4.14: Menú principal del sistema web embebido.

Vista de configuración Wi-Fi

Mediante la ruta `/wifi`, el dispositivo ofrece un formulario para actualizar las credenciales de la red local. Estas serán almacenadas en el dispositivo, cuya implementación se puede revisar en la iteración 2 en el apartado 4.4 de este capítulo. La Figura 4.15 muestra esta interfaz.



Figura 4.15: Interfaz de configuración Wi-Fi.

Vista de registro de personas

La ruta `/register` despliega una vista que combina un formulario para ingresar el nombre del usuario, rut, y correo electrónico acompañado de un stream de video en vivo desde la cámara. La Figura 4.16 lo muestra.

Registrar Persona

Nombre Completo *

Ej. Juan Perez

RUT *

Ej. 12345678-9

Email *

Ej. juan@email.com

Registrar con Huella

Ver Personas

Volver

Complete los datos y presione Registrar

Vista Previa Camara

Camara no disponible en modo offline.
Funcionalidad reservada para futuras versiones.

Template para integracion futura

Figura 4.16: Vista para el registro de personas en el dispositivo.

Vista de asistencias

Para visualizar registros de asistencia tanto locales, se implementó la vista en la ruta `/asistencias`. Esta consulta archivos JSON almacenados en SPIFFS. La Figura 4.17 lo ejemplifica.



Figura 4.17: Vista de asistencias locales almacenadas en el dispositivo.

Vista de Personas registradas

Para visualizar registros de asistencia locales, se implementó la vista en la ruta /asistencias. Esta consulta archivos JSON almacenados en SPIFFS. La Figura 4.18 lo ejemplifica.



Figura 4.18: Vista de asistencias locales almacenadas en el dispositivo.

Vista de Crear Turnos

Para la creacion de turnos, además de visualizar registros de asistencia locales, se implementó la vista en la ruta `/asistencias`. Esta consulta archivos JSON almacenados en SPIFFS. La Figura 4.19 lo ejemplifica.

Gestion de Turnos

Crear Nuevo Turno

Nombre del turno (ej. Turno Mañana)

08:00

16:00

LMOXJV

Crear Turno

Turnos Existentes

Turnos de ejemplo:

- Turno Mañana
08:00 - 16:00 LMOXJV ID: 1
- Turno Tarde
16:00 - 00:00 LMOXJV ID: 2
- Turno Noche
00:00 - 08:00 LMOXJV ID: 3
- Turno Completo
08:00 - 16:00 LMOXJV ID: 4

Volver

Figura 4.19: Vista de gestión de turnos locales almacenados en el dispositivo.

Vista de asignación de turnos

Para visualizar las asignaciones de turnos, como así realizar la accionde asignar un turno, se implementó la vista en la ruta `/asignaciones`. Esta consulta archivos JSON almacenados en SPIFFS. La Figura 4.20 lo ejemplifica.



Figura 4.20: Vista de gestión de asignaciones locales almacenadas en el dispositivo.

4.3.4 Pruebas

Para validar el funcionamiento del prototipo y comprobar la correcta integración entre los distintos módulos del sistema, se realizaron pruebas de integración y de operación en modo offline. Estas pruebas se enfocaron exclusivamente en los componentes implementados durante esta iteración, verificando su desempeño individual y conjunto. A continuación, se detallan las evaluaciones efectuadas:

- **Captura de imágenes:** Se verificó que la cámara OV2640 pudiera capturar imágenes de manera continua y estable mediante un ruta temporal `/stream`, asegurando un flujo MJPEG funcional y sin interrupciones.
- **Lectura biométrica:** Se realizaron pruebas de lectura y comparación de huellas dactilares utilizando el módulo AS608. Estas pruebas incluyeron la captura de la huella, generación del template y validación de coincidencias durante el registro y la identificación.
- **Comunicación UART:** Se comprobó que la comunicación entre el ESP32-CAM y el lector AS608 mediante el puerto UART fuese estable, sin pérdidas de paquetes ni fallos en el intercambio de comandos.
- **Carga de vistas mediante Wi-Fi:** Se validó que, al conectar el dispositivo a una red Wi-Fi externa, las mismas vistas pudieran ser cargadas sin problemas, manteniendo sus estilos, scripts y funcionalidad general.

Los resultados obtenidos permitieron confirmar que el hardware integrado (cámara y lector biométrico), las vistas web embebidas y los mecanismos de comunicación funcionan de manera coherente y estable dentro de los escenarios definidos para esta iteración.

4.4 Iteración 2: Almacenamiento local y modo offline

La segunda iteración se centra en habilitar el funcionamiento autónomo del dispositivo sin conexión a internet, integrando un sistema de almacenamiento persistente basado en SPIFFS y una interfaz web accesible mediante un Punto de Acceso (AP). Esta etapa materializa lo definido en el apartado 3.1.2 del capítulo anterior.

4.4.1 Análisis

Para el correcto funcionamiento offline del prototipo se identificaron los siguientes requerimientos:

- Almacenar localmente los datos de usuarios, turnos, asignaciones y asistencias.
- Permitir el registro de marcaciones aun cuando no exista conectividad externa.
- Garantizar persistencia de la información tras reinicios o cortes de energía.
- Habilitar la gestión completa del sistema mediante una interfaz web embebida accesible vía *Access Point*.
- Mantener compatibilidad con la futura sincronización automática en modo online.

Para cumplir estos requisitos se seleccionó el sistema de archivos **SPIFFS** y el formato **JSON** debido a su bajo peso, facilidad de manipulación y compatibilidad con las capacidades del ESP32-CAM. Adicionalmente, se definió un archivo adicional de configuración, `wifi.json`, destinado a almacenar credenciales para futuras conexiones WiFi.

4.4.2 Diseño

Considerando la arquitectura establecida, se diseñaron las siguientes estructuras de almacenamiento en formato JSON:

- `personas.json`: `id`, `nombre`, `rut`, `email`, `huella_id`, `fecha_registro`.
- `turnos.json`: `id`, `nombre`, `inicio`, `fin`, `dias`.

- **asignaciones.json**: `persona_id`, `turno_id`, `fecha_asignacion`.
- **asistencias.json**: `persona_id`, `nombre`, `tipo`, `timestamp`.
- **wifi.json**: `ssid`, `pass` (para configuraciones futuras de conectividad).

Estas estructuras permiten operar el dispositivo como una base de datos ligera totalmente local, asegurando independencia de servicios externos y continuidad operacional.

4.4.3 Implementación

En esta iteración se integraron las librerías `SPIFFS`, `ArduinoJson`, `HardwareSerial` y `Adafruit_Fingerprint`. Las dos primeras permiten gestionar archivos JSON alojados en la memoria flash del ESP32-CAM, mientras que las dos últimas habilitan la comunicación UART con el lector biométrico AS608 para el registro y verificación de huellas digitales.

El sistema también incorpora un servidor web embebido alojado en la IP local del modo *Access Point*. Este servidor entrega diferentes vistas HTML que permiten registrar usuarios, crear turnos, asignarlos, revisar asistencias y configurar parámetros operacionales.

Inicialización del sistema de archivos y persistencia

El primer componente de la implementación consiste en montar `SPIFFS` y verificar la existencia de los archivos necesarios: `personas.json`, `turnos.json`, `asignaciones.json`, `asistencias.json` y `wifi.json`. En caso de no existir, el sistema los crea con una estructura inicial vacía, asegurando que el dispositivo pueda operar incluso tras un reinicio. Este procedimiento se aprecia en la Figura 4.21.

```

void initSPIFFS() {
    if (!SPIFFS.begin(true)) {
        Serial.println("Error montando SPIFFS");
        return;
    }

    const char* files[] = {
        "/personas.json",
        "/turnos.json",
        "/asignaciones.json",
        "/asistencias.json",
        "/wifi.json"
    };

    for (auto f : files) {
        if (!SPIFFS.exists(f)) {
            File file = SPIFFS.open(f, "w");
            if (String(f) == "/wifi.json") {
                file.println("{\"ssid\":\"\\\"\", \"pass\":\"\\\"\"}");
            } else {
                file.println("[]");
            }
            file.close();
            Serial.printf("Creado %s\\n", f);
        }
    }

    Serial.println("SPIFFS inicializado");
}

```

Figura 4.21: Inicialización del sistema de archivos SPIFFS.

Este mecanismo garantiza la consistencia del estado interno del sistema y evita errores derivados de archivos faltantes o estructuras incompatibles.

Registro de usuarios con huella digital

El registro de usuarios se implementó mediante un flujo que combina la captura biométrica gestionada por el lector AS608 con el almacenamiento en `personas.json`.

El lector administra internamente modelos de huellas mediante *slots* numerados. Por ello, antes del enrolamiento es necesario identificar un slot disponible dentro del sensor, mediante la función mostrada en la Figura 4.22.

```
int encontrarSlotLibre() {  
    for (int id = 1; id < 127; id++) {  
        if (finger.loadModel(id) != FINGERPRINT_OK)  
            return id;  
    }  
    return -1;  
}
```

Figura 4.22: Búsqueda de slot libre en el lector de huellas AS608.

Una vez encontrado un slot, se ejecuta el proceso de enrolamiento, que consiste en dos capturas consecutivas de la huella para verificar coherencia y generar un modelo biométrico válido. El procedimiento implementado se muestra en la Figura 4.23.

```

bool registrarHuella(int slot) {
    Serial.printf("Iniciando enrolamiento en slot %d...\n", slot);

    int p = -1;
    int intentos = 0;

    Serial.println("Coloque el dedo...");
    while (p != FINGERPRINT_OK && intentos < 50) {
        p = finger.getImage();
        delay(100);
        intentos++;
    }

    if (p != FINGERPRINT_OK) {
        Serial.println("Timeout esperando dedo");
        return false;
    }

    p = finger.image2Tz(1);
    if (p != FINGERPRINT_OK) {
        Serial.println("Error procesando imagen 1");
        return false;
    }

    Serial.println("Retire el dedo");
    delay(2000);

    p = -1;
    intentos = 0;
    Serial.println("Coloque nuevamente el mismo dedo...");
    while (p != FINGERPRINT_OK && intentos < 50) {
        p = finger.getImage();
        delay(100);
        intentos++;
    }

    if (p != FINGERPRINT_OK) {
        Serial.println("Timeout en segunda captura");
        return false;
    }

    p = finger.image2Tz(2);
    if (p != FINGERPRINT_OK) {
        Serial.println("Error procesando imagen 2");
        return false;
    }

    p = finger.createModel();
    if (p != FINGERPRINT_OK) {
        Serial.println("Error creando modelo");
        return false;
    }

    p = finger.storeModel(slot);
    if (p == FINGERPRINT_OK) {
        Serial.println("Huella guardada correctamente");
        return true;
    }

    Serial.println("Error guardando huella");
    return false;
}

```

Figura 4.23: Procedimiento de enrolamiento biométrico en el AS608.

Luego del enrolamiento exitoso, se almacena un nuevo objeto dentro de `personas.json`, incluyendo:

- un identificador incremental,
- el nombre del usuario,
- su RUT,
- correo electrónico,
- el slot asignado en el sensor biométrico,
- un `timestamp` relativo correspondiente a la fecha de registro.

El flujo completo se encuentra encapsulado en el manejador web `handleRegisterUser()`, ilustrado en la Figura 4.24.

```

void handleRegisterUser() {
    if (!server.hasArg("name") || !server.hasArg("rut") || !server.hasArg("email")) {
        server.send(400, "text/plain", "Faltan datos requeridos");
        return;
    }

    String nombre = server.arg("name");
    String rut = server.arg("rut");
    String email = server.arg("email");

    DynamicJsonDocument doc(4096);
    JsonArray personas = loadArray("/personas.json", doc);

    // Verificar si ya existe por RUT
    for (JsonObject p : personas) {
        if (p["rut"] == rut) {
            server.send(400, "text/plain", "RUT ya registrado");
            return;
        }
    }

    int idInterno = personas.size() + 1;
    int huellaSlot = encontrarSlotLibre();

    if (huellaSlot < 0) {
        server.send(500, "text/plain", "No hay slots libres en el sensor");
        return;
    }

    Serial.printf("Registrando huella para: %s\n", nombre.c_str());
    if (!registrarHuella(huellaSlot)) {
        server.send(500, "text/plain", "Error registrando huella. Intente nuevamente.");
        return;
    }

    JsonObject p = personas.createNestedObject();
    p["id"] = String(idInterno);
    p["nombre"] = nombre;
    p["rut"] = rut;
    p["email"] = email;
    p["huella_id"] = huellaSlot;
    p["fecha_registro"] = millis() / 1000;

    saveArray("/personas.json", doc);

    server.send(200, "text/plain",
        "Usuario registrado\nNombre: " + nombre +
        "\nRUT: " + rut +
        "\nEmail: " + email +
        "\nHuella ID: " + String(huellaSlot));
}

```

Figura 4.24: Registro de usuario y almacenamiento de datos en JSON.

Registro de asistencia en modo offline

El registro de asistencia se realiza completamente en el dispositivo, sin necesidad de conexión externa. El flujo implementado consiste en:

1. Capturar una huella mediante el lector AS608.

2. Convertir la imagen y ejecutar una búsqueda biométrica.
3. Identificar al usuario asociado mediante el campo `huella_id`.
4. Verificar que cuente con un turno asignado.
5. Generar un registro en `asistencias.json` con:
 - `persona_id`,
 - nombre del usuario,
 - tipo de evento (`entrada/salida`), alternado automáticamente,
 - `timestamp` en segundos desde el inicio del sistema.

La implementación puede observarse en la Figura 4.25.

```
void handleMarcarAsistencia() {  
    Serial.println("Esperando huella...");  
  
    int p = finger.getImage();  
    if (p != FINGERPRINT_OK) {  
        server.send(500, "text/plain", "No se detecta huella");  
        return;  
    }  
  
    finger.image2Tz();  
    finger.fingerSearch();  
  
    if (finger.fingerID < 1) {  
        server.send(500, "text/plain", "Huella no encontrada");  
        return;  
    }  
  
    int huellaID = finger.fingerID;  
    Serial.printf("Huella reconocida: ID %d\n", huellaID);  
  
    String resultado = registrarAsistenciaAutomatica(huellaID);  
    server.send(200, "text/plain", resultado);  
}
```

Figura 4.25: Registro de asistencia en modo offline mediante huella digital.

```

String registrarAsistenciaAutomatica(int huellaID) {
    DynamicJsonDocument docP(4096);
    JsonArray personas = loadArray("/personas.json", docP);

    String personaId = "";
    String nombre = "";

    for (JsonObject p : personas) {
        if (p["huella_id"] == huellaID) {
            personaId = p["id"].as<String>();
            nombre = p["nombre"].as<String>();
            break;
        }
    }

    if (personaId == "") {
        return "Huella no asociada a usuario";
    }

    if (!turnoActivo(personaId)) {
        return "Usuario sin turno asignado: " + nombre;
    }

    DynamicJsonDocument docA(4096);
    JsonArray asist = loadArray("/asistencias.json", docA);

    String tipo = "entrada";
    for (int i = asist.size() - 1; i >= 0; i--) {
        JsonObject a = asist[i];
        if (a["persona_id"] == personaId) {
            tipo = (a["tipo"] == "entrada") ? "salida" : "entrada";
            break;
        }
    }

    JsonObject a = asist.createNestedObject();
    a["persona_id"] = personaId;
    a["nombre"] = nombre;
    a["tipo"] = tipo;
    a["timestamp"] = millis() / 1000;

    saveArray("/asistencias.json", docA);

    String tipoMayus = tipo;
    tipoMayus.toUpperCase();
    return tipoMayus + " registrada\nUsuario: " + nombre;
}

```

Figura 4.26: Registro de asistencia en modo offline mediante huella digital.

Gestión de datos y actualización de archivos

Para administrar los archivos JSON se desarrollaron funciones genéricas que permiten:

- cargar la estructura completa desde un archivo,
- insertar nuevos objetos dentro de un `JsonArray`,

- sobrescribir el archivo con el contenido actualizado.

Estas funciones utilizan deserialización y serialización estándar de la librería `ArduinoJson`, tal como se muestra en la Figura 4.27.

```
JsonArray loadArray(const char* path, DynamicJsonDocument &doc) {  
    File file = SPIFFS.open(path, "r");  
    String content = file.readString();  
    file.close();  
    deserializeJson(doc, content);  
    return doc.as<JsonArray>();  
}  
  
void saveArray(const char* path, DynamicJsonDocument &doc) {  
    File file = SPIFFS.open(path, "w");  
    serializeJson(doc, file);  
    file.close();  
}
```

Figura 4.27: Funciones para lectura y escritura de archivos JSON en SPIFFS.

Operación offline del dispositivo

Con el objetivo de asegurar autonomía total, el ESP32-CAM fue configurado en modo *Access Point*. El punto de acceso utiliza el SSID `ESP32-ASISTENCIA` y contraseña `12345678`, permitiendo que un administrador se conecte directamente al dispositivo para realizar todas las operaciones necesarias mediante la interfaz web embebida.

Dicha interfaz permite:

- registrar usuarios con huella,
- crear turnos,
- asignar turnos a personas,
- registrar asistencias de forma manual o automática,
- consultar personas, turnos, asignaciones y asistencias,
- gestionar la configuración WiFi.

Este mecanismo asegura que el sistema pueda operar completamente en terreno sin depender de infraestructura externa.

4.4.4 Pruebas

Las pruebas realizadas tuvieron por objetivo validar la correcta interacción entre el almacenamiento local, el lector de huellas y la interfaz web. Se evaluaron los siguientes aspectos:

- persistencia de datos tras reinicios,
- lectura y deserialización correcta de los archivos JSON,
- escritura y actualización sin pérdida de información,
- registro de asistencias offline, tanto manual como automático,
- funcionamiento del escaneo continuo del lector de huellas,
- despliegue estable de las páginas de la interfaz embebida.

Los resultados confirmaron que el prototipo puede operar en modo offline de forma estable y confiable, cumpliendo todos los objetivos planteados para esta iteración.

- persistencia de datos tras reinicios,
- lectura correcta de archivos JSON,
- escritura y actualización sin pérdida de información,
- registro de asistencias sin conexión a internet,
- verificación del funcionamiento estable del panel embebido.

Los resultados demostraron que el prototipo puede operar en modo offline sin errores, cumpliendo los objetivos de esta iteración.

4.5 Iteración 3: Comunicación y conectividad

El propósito de esta iteración consiste en implementar un backend centralizado que permita procesar la información enviada por el ESP32-CAM, administrar el reconocimiento facial, gestionar registros de trabajadores y turnos, y habilitar la comunicación bidireccional mediante MQTT. Esta etapa corresponde a una versión preliminar del sistema servidor, incorporando funcionalidades mínimas que serán extendidas en iteraciones posteriores.

4.5.1 Análisis

En esta iteración se realiza el estudio de los mecanismos de comunicación necesarios para establecer el flujo de datos entre el dispositivo ESP32-CAM y el backend del sistema, considerando tanto la operación en línea como escenarios donde la conectividad es limitada o intermitente. El análisis se centró en identificar los protocolos, tecnologías y condiciones de transmisión que permitirán asegurar que las imágenes, registros biométricos y marcaciones lleguen de manera confiable al servidor.

En primer lugar, se evaluaron los protocolos **HTTP** y **MQTT** como alternativas principales para el envío de datos desde el ESP32-CAM hacia el backend. MQTT fue seleccionado como el protocolo preferente debido a su naturaleza ligera, su eficiencia en dispositivos IoT y su capacidad de mantener sesiones persistentes con bajo consumo energético. HTTP se mantiene como protocolo complementario para la exposición de servicios REST y para procesos de sincronización posteriores entre el backend y sistemas externos. Esta combinación permite equilibrar confiabilidad, rendimiento y compatibilidad con otras plataformas.

Asimismo, se analizó la **conectividad Wi-Fi** como medio de enlace principal del dispositivo, considerando tanto el modo en línea como el funcionamiento en modo offline. En escenarios con conexión estable, el ESP32-CAM transmite imágenes al broker MQTT en tiempo real; mientras que, en situaciones de baja o nula conectividad, el dispositivo debe ser capaz de almacenar temporalmente registros y reenviarlos cuando la conexión sea restablecida. Este análisis permitió definir los mecanismos de reconexión, manejo de pérdida de paquetes y estrategias de recuperación de datos que serán abordados en el diseño e implementación.

Finalmente, se evaluó la compatibilidad del flujo de datos con la arquitectura general definida en el apartado 4.1.1, determinando que esta iteración corresponde a la habilitación de la capa de comunicación que enlaza el dispositivo con el servicio de procesamiento biométrico y el backend. Este análisis permitió delimitar claramente el alcance y las dependencias de esta etapa, asegurando coherencia con las iteraciones previas y continuidad en el desarrollo del prototipo.

4.5.2 Diseño

El diseño de esta iteración se centra en la definición detallada de los mecanismos de comunicación y conectividad necesarios para habilitar el flujo de datos entre el ESP32-CAM y el backend. Esta fase no incorpora todavía procesamiento biométrico, sino que establece las bases de transporte, recepción, almacenamiento y consulta que serán utilizadas en iteraciones posteriores.

El sistema se organiza en torno a un backend modular que cumple tres funciones

esenciales: recepción de datos mediante MQTT, exposición de servicios a través de una API REST y almacenamiento temporal de la información utilizando una base de datos de postgresSQL. Cada módulo se conecta de manera coherente, permitiendo que la transmisión de imágenes, registros y metadatos fluya correctamente desde el dispositivo hacia el servidor.

La API REST se diseña para soportar un conjunto ampliado de funcionalidades necesarias para la gestión operativa del sistema. Entre ellas se incluyen la creación de turnos, obtención de turnos individuales, listado de turnos disponibles, obtención de asignaciones de turnos y consulta de personas registradas. Estos servicios permiten estructurar y administrar la información necesaria para la sincronización y registro de datos provenientes del dispositivo. Los endpoints definidos para esta iteración incluyen:

- GET /api/personas: obtiene el listado general de personas registradas.
- POST /api/personas: registra una nueva persona en el sistema.
- GET /api/turnos: devuelve todos los turnos almacenados.
- POST /api/turnos: permite crear un nuevo turno indicando nombre, hora de inicio y hora de término.
- GET /api/turnos/<id>: entrega el detalle de un turno específico.
- GET /api/asignaciones: lista todas las asignaciones activas entre personas y turnos.
- GET /api/asignaciones/:persona_id: obtiene solo las asignaciones correspondientes a una persona.
- POST /api/asignaciones: permite asignar un turno a una persona indicando vigencia.
- POST /api/asistencias: registra una asistencia enviada desde el dispositivo.
- POST /api/asistencias/sync: recibe un conjunto de asistencias acumuladas en modo offline.

Todos estos endpoints operan sobre un modelo preliminar de base de datos que se ilustra en la figura 4.28 , la cual actúa como una base de datos simplificada para esta iteración. Las tablas definidas son: **personas**, **turnos**, **asignaciones**, **asistencias**. Cada uno contiene la información mínima necesaria para permitir operaciones de lectura, escritura y sincronización.

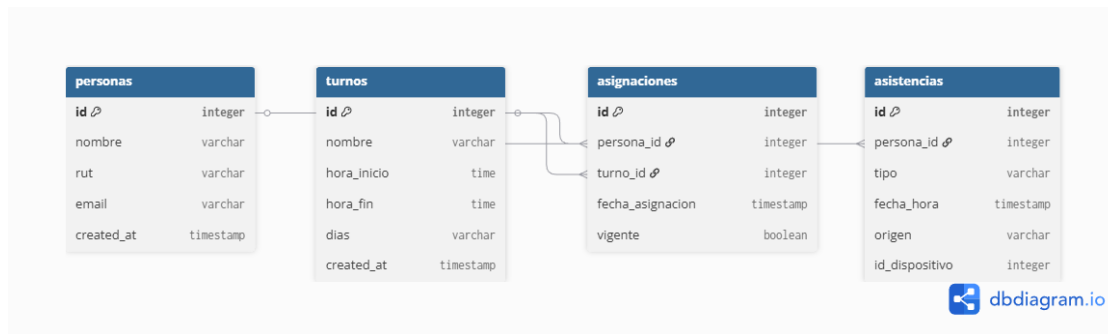


Figura 4.28: Diagrama preliminar de bases de datos

La comunicación entre el dispositivo y el backend se diseña utilizando MQTT como canal de transmisión principal. El ESP32-CAM envía imágenes fragmentadas mediante tópicos específicos (**start**, **part**, **end**), y el backend las reconstruye utilizando su módulo MQTT. Una vez almacenadas, estas imágenes serán utilizadas por módulos de procesamiento que no forman parte de esta iteración pero para la siguiente iteración. Junto con la transmisión de imágenes se contempla el envío de registros de asistencia y datos almacenados en el dispositivo, los cuales alimentan los módulos de sincronización y de registro histórico del sistema.

Finalmente, los módulos del backend se conectan mediante flujos internos que permiten que los datos recibidos por MQTT se almacenen temporalmente, se expongan mediante la API REST y se sincronicen de forma automática cuando el dispositivo opera en modo offline.

4.5.3 Implementación

La implementación de la Iteración 3 se centró en habilitar la comunicación entre el dispositivo ESP32-CAM y el backend preliminar desarrollado en Python. Esta etapa constituye la base del flujo de datos del sistema, permitiendo que el prototipo capture una imagen, la envíe al servidor y éste sea capaz de recibirla, reconstruirla y almacenarla para un posterior procesamiento en la Iteración 4. La implementación no incluye todavía mecanismos de reconocimiento facial ni validaciones biométricas, sino únicamente la capa de transporte y conectividad.

Implementación inicial de la base de datos

Para esta iteración se creó una base de datos SQL preliminar que almacena los registros generados por el backend. Se definieron las tablas personas, turnos, asignaciones, asistencias, todas con claves primarias autoincrementales (SERIAL) y restricciones referenciales mediante claves foráneas.

La tabla asignaciones incluye una restricción única sobre (persona_id, turno_id, vigente) para evitar asignaciones duplicadas en estado activo.

Se agregaron índices para optimizar consultas frecuentes, especialmente aquellas relacionadas con personas y asistencias.

Estas tablas serán extendidas en la Iteración 4 cuando se incorporen personas, encodings biométricos y registros de asistencia.

Conexión Wi-Fi del ESP32-CAM

Para permitir la operación en red, el dispositivo implementa un mecanismo de conexión automática a Wi-Fi. En esta iteración solo se requiere la capacidad de establecer conexión y reintentar en caso de falla. La función responsable de este proceso es:

```
void tryConnectWiFi() {  
    WiFi.begin(savedSSID.c_str(), savedPASS.c_str());  
    int attempts = 0;  
    while (WiFi.status() != WL_CONNECTED && attempts < 30) {  
        delay(500);  
        attempts++;  
    }  
    isOnline = (WiFi.status() == WL_CONNECTED);  
}
```

Figura 4.29: Función de conexión WiFi en el ESP32-CAM

Esta lógica asegura que el dispositivo quede habilitado para transmitir datos al backend mientras exista conectividad.

Captura de imágenes desde la cámara

Se configuró la cámara OV2640 del módulo ESP32-CAM utilizando el formato JPEG, adecuado para su transmisión por MQTT en fragmentos. La captura se realiza mediante:

```
camera_fb_t *fb = esp_camera_fb_get();  
if (!fb) {  
    Serial.println("Error al capturar imagen");  
    return;  
}
```

Figura 4.30: Inicialización de la cámara del ESP32-CAM

Esta imagen en formato binario es posteriormente codificada y enviada al broker MQTT.

Envío de imágenes mediante MQTT

Siguiendo el diseño establecido, la imagen es dividida en múltiples fragmentos para evitar desbordamiento de memoria del dispositivo y respetar los límites de tamaño del protocolo. La estructura enviada utiliza los tópicos `start`, `part` y `end`:

```
client.publish(topicStart.c_str(), "START");

String encoded = base64::encode(fb->buf, fb->len);
const size_t chunkSize = 2000;

for (size_t i = 0; i < encoded.length(); i += chunkSize) {
    String part = encoded.substring(i, i + chunkSize);
    client.publish(topicPart.c_str(), part.c_str());
}

client.publish(topicEnd.c_str(), "END")
```

Figura 4.31: Envío de imágenes por partes desde el ESP32-CAM

Este mecanismo garantiza la transferencia completa del archivo, permitiendo su reconstrucción posterior en el backend.

Reintento automático de conexión MQTT

Para mantener la comunicación estable, el dispositivo implementa un procedimiento de reconexión continua:

```
void reconnectMQTT() {  
    if (!client.connected()) {  
        if (client.connect("ESP32CAM_DEV")) {  
            client.subscribe("test/respuesta/#");  
        }  
    }  
}
```

Figura 4.32: Método para la reconexión con el MQTT

Este diseño permite que el dispositivo siga enviando imágenes incluso después de pérdidas momentáneas de red.

Recepción de imágenes en el backend

El backend escrito en Python recibe los fragmentos enviados por MQTT y reconstruye la imagen original. El proceso se realiza extrayendo la secuencia de mensajes:

```
def on_message(client, userdata, msg):  
    topic = msg.topic.split("/")[-1]  
  
    if topic == "start":  
        buffer.clear()  
  
    elif topic == "part":  
        buffer.append(msg.payload.decode())  
  
    elif topic == "end":  
        merged = "".join(buffer)  
        image_bytes = base64.b64decode(merged)  
        with open("captura.jpg", "wb") as f:  
            f.write(image_bytes)
```

Figura 4.33: Funcion para recibir imagenes en el backend.

En esta etapa la imagen únicamente se almacena; el reconocimiento facial corresponde a la Iteración 4.

Implementación de la API REST del backend

Además del canal MQTT, se habilitaron endpoints REST que permiten la consulta y recepción de datos complementarios. Su implementación es mediante Flask, de manera que se habilitaron las rutas propuestas en punto 4.5.2. Esto permite recibir todas las peticiones que se emitan desde el ESP32-CAM.

Envío de datos mediante HTTP desde el ESP32-CAM hacia la API backend

En esta iteración se implementó la capacidad del ESP32-CAM para comunicarse directamente con la API REST del backend mediante solicitudes HTTP. Esto permite que el dispositivo envíe asistencias, imágenes y sincronizaciones acumuladas, utilizando los endpoints definidos anteriormente. A continuación se listan las librerías que se utilizan para la comunicación HTTP.

- HTTPClient.h: Encargada de abrir conexiones HTTP, enviar peticiones GET/POST y recibir códigos de respuesta.
- ArduinoJson.h: Permite construir documentos JSON con estructura arbitraria para transmitir datos complejos hacia el servidor.
- base64.h: Usada para codificar imágenes en texto Base64 cuando es necesario enviarlas embebidas dentro de un JSON.
- WiFi.h: Maneja la conexión a la red, necesaria para cualquier petición HTTP.

Estas librerías permiten que el ESP32-CAM actúe como un cliente HTTP completamente funcional. La cual mediante la función 'enviarHTTP' que visualiza en la figura 4.xxx se realizan las peticiones al servidor backend variando los parámetros según sea necesario.

```

void enviarHTTP(String endpoint, bool esPOST, DynamicJsonDocument *payloadJson = nullptr) {
    HTTPClient http;
    http.begin(String(api_server) + endpoint);

    if (esPOST) {
        http.addHeader("Content-Type", "application/json");

        String jsonStr;
        if (payloadJson != nullptr) {
            serializeJson(*payloadJson, jsonStr);
        }

        int httpCode = http.POST(jsonStr);

        if (httpCode == 200) {
            Serial.println("✓ POST exitoso:");
            Serial.println(http.getString());
        } else {
            Serial.printf("✗ Error POST %s: %d\n", endpoint.c_str(), httpCode);
        }
    }
    else {
        int httpCode = http.GET();

        if (httpCode == 200) {
            Serial.println("↓ GET exitoso:");
            Serial.println(http.getString());
        } else {
            Serial.printf("✗ Error GET %s: %d\n", endpoint.c_str(), httpCode);
        }
    }

    http.end();
}

```

Figura 4.34: Metodo generico para envio de peticiones HTTP.

4.5.4 Pruebas

- estabilidad de la comunicación MQTT,
- tiempo total de transferencia,
- reconstrucción correcta del archivo en servidor,
- número de fragmentos enviados por captura.

Los resultados confirmaron la viabilidad del flujo de comunicación definido para esta iteración.

4.6 Resumen del capítulo

El capítulo 4 presenta el desarrollo del prototipo, describiendo la arquitectura física y lógica del sistema, así como la implementación práctica de cada componente. Se detalla

la integración del ESP32-CAM, el lector de huella AS608, el servidor web embebido, las vistas HTML internas, y el uso de SPIFFS para almacenamiento local.

También se documentan las primeras iteraciones del desarrollo, incluyendo la configuración inicial del hardware, el diseño de mockups de interfaz, la creación de rutas del servidor HTTP, la captura biométrica, y las pruebas realizadas para validar la comunicación, el registro de datos y la interacción entre los diferentes módulos. Este capítulo demuestra el avance concreto del prototipo y sienta las bases para las funcionalidades posteriores del sistema. Finalmente, se ejecutaron pruebas de conectividad enviando múltiples imágenes desde el ESP32-CAM hacia el backend. Para cada transmisión se verificó:

1. Ministerio del Trabajo y Previsión Social, Subsecretaría del Trabajo, Dirección del Trabajo. (2025, abril 25). Resolución 38 Exenta: Establece requisitos obligatorios y procedimiento de autorización para los sistemas electrónicos de registro y control de la asistencia y determinación de las horas de trabajo (modificada por Resolución 9453 Exenta). Biblioteca del Congreso Nacional de Chile. <https://www.bcn.cl/leychile/navegar?idNorma=1203415&idParte=10499950&idVersion=04-25>
2. Defontana Chile. (2024, mayo 9). ¿Qué es la Resolución Exenta N.º 38 en la gestión de personas? Defontana Blog. <https://www.defontana.com/blog/cl/que-es-la-resolucion-exenta-38-en-la-gestion-de-personas>
3. Redacción APD. (2025, febrero 3). Qué es un software ERP y para qué sirve: Ventajas y desventajas. APD. <https://www.apd.es/ventajas-y-desventajas-sistema-ERP>
4. Bernal, A. (2025, marzo 5). Control de asistencia integrado con ERP: Beneficios para empresas. GeoVictoria. <https://www.geovictoria.com/es-cl/blog/control-de-asistencia-integrado-con-ERP-beneficios-para-empresas>
5. Fernández, A. (2025, abril). Reloj control: Qué es, para qué sirve y qué debes tener en cuenta. Nubox Blog. <https://blog.nubox.com/empresas/reloj-control>
6. Pachon Robayo, S. (s.f.). Monitoreo de transmisión de video en vivo utilizando ESP32-CAM y Ubidots. Ubidots Centro de Ayuda. Recuperado el 17 de octubre de 2025, de <https://help.ubidots.com/es/articles/7156830-monitoreo-de-transmision-de-video-en-vivo-utilizando-ESP32-CAM-y-ubidots>
7. Amazon Web Services. (s.f.). ¿Qué es una API? Explicación de qué es una interfaz de programación de aplicaciones. Recuperado el 9 de noviembre de 2025, de <https://aws.amazon.com/es/what-is/api>

8. Ken, A. (2023, abril 10). Backend: ¿Qué es y para qué sirve? (Falta URL — agregar cuando la tengas)
9. IONOS. (s.f.). ¿Qué es un frontend? Definición y explicación. Recuperado el 28 de octubre de 2025, de <https://www.ionos.com/es-us/digitalguide/paginas-web/creacion-de-paginas-web/que-es-el-frontend>
10. Amazon Web Services. (s.f.). ¿Qué es MQTT? Explicación del protocolo MQTT. Recuperado el 6 de octubre de 2025, de <https://aws.amazon.com/es/what-is/mqtt>
11. Erickson, J. (2024, abril 4). ¿Qué es JSON? Oracle América Latina. Recuperado el 14 de noviembre de 2025, de <https://www.oracle.com/latam/database/what-is-json>
12. Last Minute Engineers. (s.f.). Empezando con ESP32-CAM: Guía para principiantes. Recuperado el 19 de octubre de 2025, de <https://lastminuteengineers.com/getting-started-with-ESP32-CAM>
13. UNIT Electronics. (s.f.). Cómo utilizar el lector de huella dactilar AS608. Recuperado el 22 de noviembre de 2025, de <https://blog.uelectronics.com/tarjetas-desarrollo/como-utilizar-el-lector-de-huella-dactilar-AS608>
14. MDN Web Docs. (s.f.). Generalidades del protocolo HTTP. Recuperado el 30 de octubre de 2025, de <https://developer.mozilla.org/es/docs/Web/HTTP/Guides/Overview>
15. Paessler GmbH. (s.f.). ¿Qué es MQTT? – IT Explained. Recuperado el 2 de noviembre de 2025, de <https://www.paessler.com/es/it-explained/mqtt>
16. Docker Inc. (s.f.). ¿Qué es un contenedor? Recuperado el 21 de octubre de 2025, de <https://www.docker.com/resources/what-container>
17. Sunny Valley Cyber Security Inc. (2025, febrero 3). Definición, tipos y ejemplos de servidor de base de datos. Zenarmor. Recuperado el 10 de noviembre de 2025, de <https://www.zenarmor.com/docs/network-basics/what-is-database-server>
18. GeoVictoria. (s.f.). Dispositivos GeoVictoria. Recuperado el 12 de octubre de 2025, de <https://info.geovictoria.com/es-co/dispositivos>
19. ZKTeco Latinoamérica. (s.f.). Tiempo y asistencia. Recuperado el 5 de noviembre de 2025, de <https://zktecolatinoamerica.com/categoria-producto/tiempo-y-asistencia>
20. Talana. (s.f.). Software de asistencia de personal en Chile. Recuperado el 7 de noviembre de 2025, de <https://web.talana.com/software-de-asistencia-de-personal>

21. Buk. (s.f.). Control de asistencia: Gestiona tiempos y alertas. Recuperado el 26 de octubre de 2025, de <https://www.buk.cl/productos/administracion/control-de-asistencia>
22. Defontana. (s.f.). Software y sistema ERP. Recuperado el 18 de noviembre de 2025, de <https://www.defontana.com/cl>
23. Softland Inversiones S.L. (s.f.). Softland Chile: Sistema ERP, contable y de administración. Recuperado el 9 de octubre de 2025, de <https://www.softland.cl>
24. Drumond, C. (s.f.). Scrum: Qué es, cómo funciona y cómo empezar. Atlassian. Recuperado el 3 de noviembre de 2025, de <https://www.atlassian.com/es/agile/scrum>
25. Teach-ICT. (s.f.). Prototipado evolutivo. Recuperado el 22 de octubre de 2025, de https://www.teach-ict.com/as_a2_ict_new/ocr/A2_G063/331_systems_cycle/prototyping_R
26. GeeksforGeeks. (2025, julio 11). Modelo de procesos incrementales – Ingeniería de software. Recuperado el 20 de noviembre de 2025, de <https://www.geeksforgeeks.org/software-engineering/software-engineering-incremental-process-model>
27. Pressman, R. S., & Maxim, B. R. (2021). Ingeniería de software: Un enfoque práctico (9.^a ed.). McGraw-Hill Interamericana.
28. IBM Corporation. (2025). ¿Qué son las pruebas de integración? IBM Think. <https://www.ibm.com/es-es/think/topics/integration-testing>
29. International Organization for Standardization. (2018). ISO 9241-11: Ergonomics of human-system interaction — Part 11: Usability: Definitions and concepts.
30. Nielsen, J. (1993). Usability engineering. Morgan Kaufmann.
31. Brooke, J. (1996). SUS: A “quick and dirty” usability scale. En P. W. Jordan et al. (Eds.), Usability evaluation in industry (pp. 189–194). Taylor & Francis.
32. Bangor, A., Kortum, P. T., & Miller, J. T. (2008). An empirical evaluation of the System Usability Scale. *International Journal of Human-Computer Interaction*, 24(6), 574–594.
33. Microsoft. (2025, julio 10). Introducción a Internet de las cosas (IoT) de Azure. Microsoft Learn. <https://learn.microsoft.com/es-es/azure/iot/iot-introduction>
34. Atlassian. (s.f.). Reliability vs. availability: Key metrics for system performance. Recuperado el 11 de noviembre de 2025, de <https://www.atlassian.com/incident-management/kpis/reliability-vs-availability>

35. Perry, W. E. (2006). Effective methods for software testing (3.^a ed.). Wiley.
36. Kaspersky Lab. (2025). ¿Qué es la biometría? Definición y tipos. <https://latam.kaspersky.com/center/definitions/biometrics>
37. Espressif Systems. (2025). SPIFFS — SPI Flash File System. <https://docs.espressif.com/projects/idf/en/stable/esp32/api-reference/storage/spiffs.html>
38. Amazon Inc. (2025). What is a RESTful API? <https://aws.amazon.com/what-is/restful-api>
39. Red Hat Inc. (2025). ¿Qué son los microservicios? <https://www.redhat.com/es/topics/microserv>
40. EMQX Technologies. (2025). The ultimate guide to MQTT brokers. <https://www.emqx.com/en/ultimate-guide-to-mqtt-broker-comparison>